

POLITECNICO
MILANO 1863

Numerical Analysis for Machine Learning

Neural-Gaussian Surrogates for Rocket Aerodynamics

Riccardo Bellotti
Marco D'Agostino

Supervisor: Prof. Edie Miglio

Academic Year 2025/2026

Abstract

This work investigates the development and evaluation of hybrid surrogate models for rocket aerodynamics within the framework of the Numerical Analysis for Machine Learning (NAML) course. In particular, we propose and analyze hybrid architectures combining Multi-Layer Perceptrons (MLPs) and Gaussian Processes (GPs), aiming to exploit the complementary strengths of deep learning and probabilistic modeling.

The proposed models are validated on standard benchmark datasets and further assessed on the HYDDB aerodynamic database, generated through an automated CFD pipeline on a fixed rocket geometry where only the airbrake opening varies across samples.

The dataset covers a wide range of flight conditions (Mach, angle of attack, roll angle, altitude) combined with the variable airbrake aperture.

Results show that the hybrid MLP–GP variants offer competitive predictive accuracy and informative uncertainty bands relative to standalone approaches.

Contents

1 Introduction **4**

2 Models architecture **4**
2.1 Multi-Layer Perceptrons 4
2.1.1 Weight Initialization Strategy 4
2.2 Gaussian Processes 5
2.3 Hybrid MLP–GP Models 6
2.3.1 Non-Stationary Gaussian Processes 6
2.3.2 Deep Kernel Learning 6
2.3.3 Comparison of the Two Strategies 6

3 Training procedure and hyperparameters optimization **7**
3.1 Training Procedure 7
3.2 Training Stop Policy 8
3.3 Hyperparameter Optimization 8

4 Standard datasets benchmark **9**

5 The HYDDB Aerodynamic Database **10**
5.1 Physical and Computational Problem 10
5.2 Dataset Typology 10
5.3 Automated CFD Pipeline 10
5.4 Sampling Strategy 11

6 Results and Analysis on HYDDB **11**
6.1 Evaluation Metrics 11
6.2 Quantitative Results 12
6.3 Sampling-Plan Coverage 12
6.4 Uncertainty Reliability 12
6.5 Discussion 13

7 Conclusions and Future Work **13**
7.1 Conclusions 13
7.2 Future Work 14

A ISA and Reynolds Number **14**

B Reference Rocket Geometry **15**
B.1 Global Rocket Geometry 15
B.2 Fin Geometry 16
B.3 Airbrake System 16
B.4 Use within HYDDB 18

C Mesh Generation **18**
C.1 Computational Domain 18
C.2 Mesh Generation Strategy 18
C.3 Boundary Layer Resolution 18
C.4 Surface Mesh Details 18
C.5 Mesh Size and Case-to-Case Variability 18

D CFD simulation setup **22**

D.1 Solver Configuration 22
D.2 Turbulence Modeling 22
D.3 Fluid Properties 22
D.4 Boundary Conditions 22
D.5 Representative Flow Field Visualization 22

E CFD simulation setup validation **23**
E.1 Validation Benchmark 23
E.2 Numerical Setup 23
E.3 Validation Matrix 23
E.4 Validation Metrics 24
E.5 Results at $M = 0.3$ 24
E.6 Results at $M = 0.5$ 24
E.7 Results at $M = 0.8$ 24
E.8 Results at $M = 1.0$ 24
E.9 Overall Assessment 24
E.10 Conclusions 31

1 Introduction

The accurate prediction of aerodynamic coefficients is a critical component in the design, analysis, and optimization of aerospace systems. Computational Fluid Dynamics (CFD) simulations provide reliable results but are computationally expensive, making them impractical for tasks requiring many evaluations, such as design space exploration or optimization. To address this limitation, surrogate models are commonly employed to approximate the underlying physical function with significantly reduced computational cost.

This report investigates and compares two prominent surrogate modeling approaches: Multi-Layer Perceptrons (MLPs) and Gaussian Processes (GPs).

MLPs, as flexible data-driven models, are capable of capturing highly nonlinear relationships in large datasets, while GPs offer a probabilistic framework that naturally quantifies prediction uncertainty and performs well in low-data regimes [8]. Despite their individual strengths, both approaches exhibit limitations when used independently.

To overcome these challenges, we explore hybrid methodologies that combine Neural Networks and Gaussian Processes. In particular, we focus on approaches such as non-stationary Gaussian Processes and deep kernel regression, where Neural Networks are used to enhance the representational power of kernel-based methods [18]. These hybrid models aim to leverage the scalability and expressiveness of MLPs together with the robustness and uncertainty quantification of GPs.

The proposed methodologies are first evaluated on benchmark datasets to highlight their comparative performance under controlled conditions. Subsequently, they are applied to a CFD-generated dataset specifically developed for this project, involving the prediction of aerodynamic coefficients of a variable-geometry rocket under varying flight conditions. Special attention is given to the impact of dataset size, comparing model behavior in both data-scarce and data-rich scenarios.

2 Models architecture

This section presents the surrogate modeling strategies investigated in this work. Three different model families are considered:

- Multi-Layer Perceptrons (MLPs);
- Gaussian Processes (GPs);
- hybrid MLP–GP approaches.

2.1 Multi-Layer Perceptrons

Multi-Layer Perceptrons (MLPs) are fully connected feedforward Neural Networks widely employed for nonlinear regression tasks. Given an input vector $\mathbf{x} \in \mathbb{R}^d$, an L-layer MLP defines a nonlinear mapping:

$$\mathbf{y} = f(\mathbf{x}; \theta) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}), \quad (1)$$

where each layer is expressed as:

$$f_\ell(\mathbf{h}) = \phi_\ell(\mathbf{W}_\ell \mathbf{h} + \mathbf{b}_\ell). \quad (2)$$

Here, $\mathbf{W}_\ell$ and $\mathbf{b}_\ell$ denote weights and biases, while $\phi_\ell(\cdot)$ represents the activation function.

In this work, fully connected architectures with varying depth and width are considered. The output layer adopts a linear activation function, which is more suitable for continuous regression tasks involving aerodynamic coefficients.

A systematic investigation of hidden-layer activation functions is conducted, as the choice of activation directly influences both the expressive capacity of the network and the conditioning of the optimization landscape. Five activations are considered: ReLU, Leaky ReLU, SELU, Tanh, and Mish [14]. A dedicated weight initialization strategy is paired with each.

2.1.1 Weight Initialization Strategy

The initialization of Neural Network weights is critical for stable and efficient training. Improper initialization leads to vanishing or exploding gradients, severely hindering optimization and limiting network depth. The guiding principle is to preserve activation variance across layers, ensuring well-conditioned signal propagation throughout the network.

In practice, weights are sampled as $w \sim \mathcal{N}(0, \sigma_w^2)$, where σ_w^2 is the activation-specific variance prescribed below. The zero-mean constraint ensures centered activations across layers; the variance prescription prevents signal collapse or explosion. For a linear pre-activation $z^{(l)} = \sum_{j=1}^{n_{\text{in}}} w_j a_j^{(l-1)}$, it can be shown that

$$\text{Var}(z^{(l)}) = n_{\text{in}}\, \sigma_w^2\, \text{Var}(a^{(l-1)}) \quad (3)$$

under the assumptions that weights and activations are i.i.d. and mutually independent with zero mean. Setting $\mathrm{Var}(z^{(l)}) = \mathrm{Var}(a^{(l-1)})$ yields the canonical constraint $\sigma_w^2 = 1/n_{\text{in}}$, which each scheme below modifies to account for the nonlinearity of the chosen activation function.

Biases are initialized to zero, $\mathbf{b}^{(l)} = \mathbf{0}$. Unlike weights, biases are additive offsets independent across neurons; they do not require random initialization.

Each activation function requires a specific initialization strategy:

ReLU and Leaky ReLU: He (Kaiming) normal initialization [9] with weight variance:

$$\sigma_w^2 = \frac{2}{n_{\text{in}}} \tag{4}$$

The factor of 2 compensates for ReLU zeroing approximately half of activations, preserving gradient flow across layers. For Leaky ReLU, the gain is adjusted to $\sqrt{2/(1+\alpha^2)}$, where α is the negative slope.

SELU (Scaled Exponential Linear Unit): LeCun normal initialization [12] with variance:

$$\sigma_w^2 = \frac{1}{n_{\text{in}}} \tag{5}$$

SELU is designed for self-normalizing networks [11], maintaining zero mean and unit variance across layers without batch normalization; the reduced variance reflects this built-in stabilization.

Tanh: Xavier (Glorot) normal initialization [7] with variance:

$$\sigma_w^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}} \tag{6}$$

The fan-in/fan-out symmetry balances forward and backward signal propagation, with a gain factor ≈ 1.2 accounting for Tanh's nonlinear variance compression.

Mish: He (Kaiming) normal initialization with ReLU-family gain:

$$\sigma_w^2 = \frac{2}{n_{\text{in}}} \tag{7}$$

Mish lacks a closed-form gain derivation; the ReLU heuristic is adopted based on its asymptotic equivalence to ReLU for large positive inputs.

2.2 Gaussian Processes

Gaussian Processes (GPs) provide a probabilistic framework for nonlinear regression by modeling the latent target function as a stochastic process:

$$f(\mathbf{x}) \sim \mathcal{GP}\Big(m(\mathbf{x}),\, \sigma^2 R(\mathbf{x}, \mathbf{x}')\Big), \tag{8}$$

where $m(\mathbf{x}) = \sum_{j=0}^{q} \beta_j f_j(\mathbf{x})$ is a polynomial trend of degree q (treated as a hyperparameter, see Tab. 1) and $R(\mathbf{x}, \mathbf{x}')$ is the spatial correlation kernel. In this work, Universal Kriging is adopted using a generalized separable power-exponential kernel [6]:

$$R(\mathbf{x}, \mathbf{x}') = \exp\left(-\sum_{k=1}^{d} \theta_k \, |x_k - x_k'|^{p_k}\right), \tag{9}$$

where $\theta_k > 0$ are inverse lengthscales and $p_k \in [1, 2]$ controls the smoothness of the response surface. To account for numerical noise and improve conditioning, a nugget regularization term is introduced:

$$\mathbf{C} = \mathbf{R} + \lambda \mathbf{I}, \tag{10}$$

where $\lambda \geq 0$ represents the nugget effect.

Conditioning the prior GP on n observed data points yields closed-form predictive expressions. Let $\mathbf{r}^* = \mathbf{r}(\mathbf{x}^*)$ denote the cross-correlation vector between the prediction point and the training set, and $\mathbf{f}^* = \mathbf{f}(\mathbf{x}^*)$ the vector of trend basis functions evaluated at $\mathbf{x}^*$. In the classical Universal Kriging formulation, the trend coefficients are estimated via Generalized Least Squares (GLS):

$$\hat{\boldsymbol{\beta}} = \left(\mathbf{F}^\top \mathbf{C}^{-1} \mathbf{F}\right)^{-1} \mathbf{F}^\top \mathbf{C}^{-1} \mathbf{y}, \tag{11}$$

where $\mathbf{F} \in \mathbb{R}^{n \times q}$ is the regression matrix of trend basis functions at the training points.

In the present implementation, however, the coefficients of the polynomial trend are treated as trainable parameters and optimized jointly with the remaining hyperparameters through marginal log-likelihood maximization.

The posterior mean and predictive variance at $\mathbf{x}^*$ are then given by:

$$\hat{y}(\mathbf{x}^*) = \mathbf{f}^{*\top} \hat{\boldsymbol{\beta}} + \mathbf{r}^{*\top} \mathbf{C}^{-1} \left(\mathbf{y} - \mathbf{F}\hat{\boldsymbol{\beta}}\right), \tag{12}$$

$$s^2(\mathbf{x}^*) = \sigma^2 \left(1 - \mathbf{r}^{*\top} \mathbf{C}^{-1} \mathbf{r}^* + \mathbf{u}^{*\top} \left(\mathbf{F}^\top \mathbf{C}^{-1} \mathbf{F}\right)^{-1} \mathbf{u}^*\right),$$

where $\mathbf{u}^* = \mathbf{F}^\top \mathbf{C}^{-1}\mathbf{r}^* - \mathbf{f}^*$ captures the uncertainty contribution from trend estimation. This formulation enables uncertainty quantification alongside deterministic predictions. The kernel hyperparameters $\{\theta_k, p_k\}$ and the nugget λ are estimated through Maximum Likelihood Estimation (MLE) [17], maximizing the Gaussian log-likelihood of the observed training data.

2.3 Hybrid MLP–GP Models

Although MLPs offer strong expressive power and scalability to large datasets, they do not naturally provide uncertainty estimates. Conversely, Gaussian Processes provide a rigorous probabilistic formulation [17] but may suffer from limited flexibility when highly non-stationary behaviors are present or for large dataset problems. To combine the strengths of both methodologies, hybrid MLP–GP approaches are investigated. In particular, two different formulations are considered:

- Non-Stationary Gaussian Processes;
- Deep Kernel Learning (DKL).

2.3.1 Non-Stationary Gaussian Processes

Standard GP formulations assume stationarity, meaning that the statistical properties of the process remain invariant across the input domain. This assumption is often restrictive in aerodynamic applications characterized by localized nonlinearities and regime-dependent behaviors.

To address this limitation, kernel hyperparameters are made input-dependent through a neural parameterization. Specifically, an MLP is employed to generate spatially varying amplitude and lengthscale fields:

$$\sigma(\mathbf{x}), \qquad \boldsymbol{\ell}(\mathbf{x}) = [\ell_1(\mathbf{x}), \ldots, \ell_d(\mathbf{x})]. \tag{13}$$

The resulting covariance function follows a generalized Gibbs formulation [15]:

$$k(\mathbf{x}, \mathbf{x}') = \sigma(\mathbf{x})\,\sigma(\mathbf{x}') \prod_{k=1}^{d} \alpha_k(\mathbf{x}, \mathbf{x}')\, e^{-\Delta(\mathbf{x}, \mathbf{x}')}, \tag{14}$$

where $\Delta(\mathbf{x}, \mathbf{x}') = \sum_{k=1}^{d} \delta_k(\mathbf{x}, \mathbf{x}')$ is the weighted Euclidean distance and

$$\alpha_k(\mathbf{x}, \mathbf{x}') = \sqrt{\frac{2\,\ell_k(\mathbf{x})\,\ell_k(\mathbf{x}')}{\ell_k(\mathbf{x})^2 + \ell_k(\mathbf{x}')^2}} \tag{15}$$

is the geometric blending factor between local lengthscales. The effective position-dependent correlation length is defined as:

$$\bar{\ell}_k(\mathbf{x}, \mathbf{x}') = \sqrt{\frac{\ell_k(\mathbf{x})^2 + \ell_k(\mathbf{x}')^2}{2}}, \tag{16}$$

and the locally adapted distance metric as:

$$\delta_k(\mathbf{x}, \mathbf{x}') = \theta_k \left(\frac{|x_k - x_k'|}{\bar{\ell}_k(\mathbf{x}, \mathbf{x}')} \right)^{p_k}. \tag{17}$$

The amplitude term $\sigma(\mathbf{x})\,\sigma(\mathbf{x}')$ accounts for the spatially varying variance of the process. In this formulation, the MLP acts as a *parameterizer*, learning spatially varying amplitude and lengthscale fields directly from data. This allows the GP to adapt its covariance structure locally, effectively capturing heteroscedasticity and varying smoothness across the input domain.

2.3.2 Deep Kernel Learning

Deep Kernel Learning (DKL) [18] combines neural feature extraction with kernel-based probabilistic regression. A Neural Network $f(\mathbf{x})$ is first employed to map the original input space into a latent representation:

$$\mathbf{z} = f(\mathbf{x}), \qquad \mathbf{z} \in \mathbb{R}^{d_{\text{latent}}}. \tag{18}$$

A stationary GP kernel is then applied within the learned latent space:

$$k_{\text{DKL}}(\mathbf{x}, \mathbf{x}') = k_{\text{base}}(f(\mathbf{x}), f(\mathbf{x}')), \tag{19}$$

where k_{base} is the power-exponential kernel described in Section 2. This formulation can be interpreted as learning a data-driven similarity metric, where the feature extractor implicitly defines the notion of proximity between inputs. The latent dimensionality d_{latent} constitutes an additional architectural degree of freedom: it is typically kept small to preserve the tractability of the GP regression stage, but must be sufficient to retain the relevant structure of the input space.

2.3.3 Comparison of the Two Strategies

The non-stationary GP and DKL approaches differ in how the neural component interacts with the GP. In the non-stationary formulation, the MLP *parameterizes* the kernel directly, modulating amplitude and lengthscales point-wise while the GP

still operates in the original input space. In DKL, the MLP acts as a *feature extractor*, transforming the input space so that a simpler stationary kernel suffices in the latent domain. Both strategies retain the probabilistic nature of the GP, providing predictive uncertainty estimates alongside the mean prediction. However, the coupling between Neural Network weights and GP hyperparameters introduces a more complex optimization landscape compared to either model trained in isolation.

3 Training procedure and hyperparameters optimization

3.1 Training Procedure

The training of the surrogate models is performed by minimizing a regression objective over the available dataset. For purely neural architectures (MLPs), the adopted loss function is the Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{c}_i - \hat{\mathbf{c}}_i\|^2, \tag{20}$$

where $\mathbf{c}_i$ denotes the true aerodynamic coefficients and $\hat{\mathbf{c}}_i$ the corresponding model predictions.

Conversely, Gaussian Process models and hybrid MLP–GP formulations are trained through minimization of the Negative Log-Likelihood (NLL):

$$\mathcal{L}_{\text{NLL}} = -\log p(\mathbf{y} \mid \Theta), \tag{21}$$

where Θ denotes the complete set of trainable parameters, including kernel hyperparameters and Neural Network weights.

Assuming:

$$\mathbf{y} \sim \mathcal{N}(\mathbf{F}\boldsymbol{\beta}, \sigma^2\mathbf{C}), \tag{22}$$

the resulting objective function can be written as:

$$\mathcal{L}_{\text{NLL}} = \frac{n}{2}\log(\sigma^2) + \frac{1}{2}\log|\mathbf{C}| + \frac{1}{2\sigma^2}\boldsymbol{\epsilon}^T\mathbf{C}^{-1}\boldsymbol{\epsilon}, \tag{23}$$

where $\boldsymbol{\epsilon} = \mathbf{y} - \mathbf{F}\boldsymbol{\beta}$ denotes the residuals. up to additive constants independent of the trainable parameters.

Training all three architectures relies on *automatic differentiation*: trainable parameters are registered as `torch.nn.Parameter` objects, making them leaf nodes of PyTorch's dynamic computational graph. During the forward pass, every arithmetic operation involving these tensors is recorded by the *autograd* engine, which constructs a directed acyclic graph (DAG) of differentiable primitives. A single call to `loss.backward()` then traverses the DAG in reverse topological order, accumulating gradients via the chain rule without requiring any user-defined Jacobian:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \sum_j \frac{\partial \mathcal{L}}{\partial h_j}\frac{\partial h_j}{\partial \theta_k}, \tag{24}$$

where h_j denotes any intermediate quantity in the forward computation that depends on θ_k. This is particularly convenient for the MLP–GP variants, where the gradient flows seamlessly from the negative log-likelihood through both the kernel hyperparameters and the MLP feature extractor in a single backward pass, with no manually derived Jacobian.

This joint training strategy and the kernel parameterization follow established practices in deep kernel learning [18]. For purely neural models (**MLP**), $\Theta = \{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L}$ collects weights and biases across all layers, and the gradient of the MSE loss flows back through the network via standard backpropagation.

For **GP** models, the trainable set $\Theta = \{\boldsymbol{\theta}_k, \sigma^2, \lambda\}$ comprises the kernel hyperparameters. Autograd differentiates through the Cholesky factorization $\mathbf{C} = \mathbf{L}\mathbf{L}^\top$, which is used for numerically stable evaluation of $\log|\mathbf{C}|$ and $\mathbf{C}^{-1}\boldsymbol{\epsilon}$ in eq. (23), so that $\nabla_{\boldsymbol{\theta}_k}\mathcal{L}_{\text{NLL}}$ is computed without explicit inversion of $\mathbf{C}$.

The **MLP–GP** hybrid presents the most involved gradient path. Here the MLP acts as a feature extractor:

$$\mathbf{z}_i = f_{\boldsymbol{\theta}_{\text{MLP}}}(\mathbf{x}_i) \in \mathbb{R}^{d_z}, \tag{25}$$

and the kernel is evaluated on the learned representations $\mathbf{z}_i$ rather than on the raw inputs $\mathbf{x}_i$. Consequently, the NLL in eq. (23) depends on $\boldsymbol{\theta}_{\text{MLP}}$ through the kernel matrix:

$$\frac{\partial \mathcal{L}_{\text{NLL}}}{\partial \boldsymbol{\theta}_{\text{MLP}}} = \frac{\partial \mathcal{L}_{\text{NLL}}}{\partial \mathbf{C}}\frac{\partial \mathbf{C}}{\partial \mathbf{z}}\frac{\partial \mathbf{z}}{\partial \boldsymbol{\theta}_{\text{MLP}}}, \tag{26}$$

so that a single backward pass jointly updates both MLP weights $\boldsymbol{\theta}_{\text{MLP}}$ and kernel hyperparameters $\boldsymbol{\theta}_k$ through the same computational graph, without requiring alternating or nested optimization loops. Since the two parameter groups operate on different

scales and exhibit different curvatures, they are assigned separate learning rates η_{MLP} and η_{GP} within a single AdamW optimizer, as reported in Table 1.

Optimization of all trainable parameters is carried out using the AdamW algorithm [13], which extends the standard Adam optimizer [10] by decoupling weight decay regularization from the gradient-based update step. At each iteration, AdamW computes exponentially weighted estimates of the first and second moments of the gradients:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{27}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{28}$$

where $g_t = \nabla_\theta \mathcal{L}_t$ is the gradient of the loss at iteration t, and $\beta_1 = 0.9$, $\beta_2 = 0.999$ are fixed to their default values throughout all experiments.

After bias correction, the parameter update is performed according to:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t, \tag{29}$$

where η is the learning rate and λ the weight decay coefficient.

3.2 Training Stop Policy

To ensure robust and consistent convergence behavior across all surrogate models, a unified stopping policy is adopted and consistently applied also to GP-based and hybrid MLP–GP models.

Training is terminated whenever *either* of the following conditions is satisfied:

1. **Training loss convergence:**
   convergence is detected when the variation of the training loss between two consecutive iterations satisfies:

   $$|\mathcal{L}_{t-1} - \mathcal{L}_t| \le \epsilon_{\text{abs}} + \epsilon_{\text{rel}} |\mathcal{L}_{t-1}|, \tag{30}$$

   for a predefined number of consecutive iterations (*train patience*).
   This criterion is analogous to the stopping condition commonly employed in nonlinear least-squares optimization algorithms.
2. **Validation early stopping:**
   following the strategy proposed by Prechelt [16], the validation loss is continuously monitored and an improvement is considered significant only if:

   $$\mathcal{L}_{\text{best}} - \mathcal{L}_{\text{val}} > \epsilon_{\text{abs}} + \epsilon_{\text{rel}} |\mathcal{L}_{\text{best}}|. \tag{31}$$

   Training is interrupted after a predefined number of consecutive non-improving validation iterations (*validation patience*). Note that, while the GP-family training loss is the negative log-likelihood, the validation criterion $\mathcal{L}_{\text{val}}$ is the held-out mean squared error (denormalised back to the original output scale) for all architectures. This decouples the training objective from a comparable, model-agnostic stopping signal.

This dual stopping criterion allows balancing convergence detection and generalization capability, preventing both premature stopping and unnecessary overtraining.

The introduction of early stopping is particularly important for hybrid MLP–GP architectures, especially in the non-stationary formulation, where the increased flexibility induced by neural parameterization can easily lead to overfitting and unstable local kernel adaptations.

3.3 Hyperparameter Optimization

The hyperparameter optimization process is performed using the *Optuna* framework with a Tree-structured Parzen Estimator (TPE) sampler [4].

The TPE algorithm is a sequential Bayesian optimization strategy that models promising and non-promising regions of the hyperparameter space through adaptive probability density estimators. Instead of directly modeling the objective function, TPE constructs two probability distributions:

$$l(\mathbf{z}) = p(\mathbf{z} \mid y < y^*), \qquad g(\mathbf{z}) = p(\mathbf{z} \mid y \ge y^*),$$

where $\mathbf{z}$ denotes the hyperparameter vector and y^* is a threshold corresponding to the best-performing trials. New candidate configurations are sampled by maximizing the ratio:

$$\frac{l(\mathbf{z})}{g(\mathbf{z})},$$

thereby favoring regions of the search space associated with lower objective values while still preserving exploration capabilities. For each surrogate model, a total of 50 optimization trials are executed.

The complete set of optimized hyperparameters is summarized in Table 1.

For GP-based models, the kernel hyperparameters sampled by Optuna are interpreted as initialization

Table 1: Hyperparameters optimized via TPE–Optuna. ✓ = applies, – = not applicable, † = DKL only.

Group	Param.	MLP	GP	MLP–GP
Arch.	N_L	✓	–	✓
	N_H	✓	–	✓
	ϕ	✓	–	✓
	d_z	–	–	†
Kernel	θ_k	–	✓	✓
	p_k	–	✓	✓
	σ^2	–	✓	✓
	λ	–	✓	✓
	q	–	✓	✓
Optim.	η_{MLP}	✓	–	✓
	η_{GP}	–	✓	✓
	γ	✓	–	✓

N_L number of hidden layers
N_H neurons per hidden layer
ϕ activation function
d_z latent output width (DKL)
θ_k inverse lengthscales initialization
p_k power exponents initialization
σ^2 process variance initialization
λ noise / nugget initialization
q trend polynomial degree
η learning rate
γ MLP weight decay

values for the trainable kernel parameters. The actual kernel coefficients are subsequently refined during gradient-based optimization through minimization of the NLL objective.

Once the optimal hyperparameter configuration is identified, the corresponding model is retrained and evaluated over 5 independent runs using different random seeds. The random seed simultaneously affects dataset partitioning, neural weight initialization, and stochastic optimization trajectories. The final reported metrics are therefore expressed as mean and standard deviation across the independent runs.

4 Standard datasets benchmark

Before evaluating the surrogate models on the HYDDB aerodynamic database, a preliminary benchmark analysis is conducted on standard regression datasets obtained from the UCI Machine Learning Repository [5]. Only datasets of moderate size are considered in order to maintain the computational feasibility of exact Gaussian Process regression methods.

The benchmark includes the following datasets:

- SkillCraft1 Master Table Dataset (SkillCraft);
- Concrete Compressive Strength (CCS);
- Airfoil Self-Noise Dataset (Airfoil);
- Yacht Hydrodynamics Dataset (YachtHydro).

For all datasets, the same experimental protocol is adopted:

- 85% of the samples are employed for training and validation;
- 15% of the samples are reserved for testing;
- 20% of the training subset is used for validation and early stopping;
- hyperparameters are optimized using Optuna with a TPE sampler;
- each optimized configuration is evaluated over 5 independent runs using different random seeds.

The benchmark includes the following surrogate model families:

- Multi-Layer Perceptrons (MLPs);
- Gaussian Processes (GPs);
- hybrid MLP–GP models:
  - non-stationary GP;
  - Deep Kernel Learning (DKL).

Table 2 summarizes the main characteristics of the benchmark datasets (number of samples N and input features d) together with the predictive performance obtained by the considered surrogate models.

Table 2: Benchmark datasets characteristics and surrogate model performance (mean ± std, 5 runs).

Dataset	N	d	Model	RMSE
SkillCraft	3338	19	MLP	0.254 ± 0.008
			GP	0.243 ± 0.007
			NSGP	0.249 ± 0.006
			DKL	0.252 ± 0.008
CCS	1030	8	MLP	4.792 ± 0.492
			GP	4.094 ± 0.478
			NSGP	4.047 ± 0.458
			DKL	4.664 ± 0.464
Airfoil	1503	5	MLP	1.455 ± 0.146
			GP	1.310 ± 0.257
			NSGP	1.280 ± 0.190
			DKL	1.240 ± 0.155
YachtHydro	308	6	MLP	0.737 ± 0.186
			GP	0.487 ± 0.088
			NSGP	0.485 ± 0.188
			DKL	0.407 ± 0.106

Overall, the benchmark analysis provides a preliminary comparison of the different surrogate modeling approaches on heterogeneous regression problems before their application to the HYDDB aerodynamic database.

5 The HYDDB Aerodynamic Database

5.1 Physical and Computational Problem

The primary objective of this project is to estimate the aerodynamic coefficients of a rocket configuration with high accuracy, while maintaining a computational cost compatible with extensive system-level simulation loops (e.g., 6-DOF trajectory integration). The aerodynamic state is in general described by a multi-output vector:

$$\mathbf{c}(\mathbf{x}) = [C_A, C_Y, C_N, C_l, C_m, C_n]^T,$$

where the components represent the axial force, side force, normal force, rolling moment, pitching moment, and yawing moment coefficients, respectively. In the present work, attention is restricted to the axial force coefficient C_A as the sole output of interest, leaving the extension to the full multi-output formulation to future work.

The system is parameterized by a 5-dimensional input vector:

$$\mathbf{x} = [M, \alpha, \phi, h, b],$$

where M is the Mach number, α is the total angle of attack, ϕ is the aerodynamic roll angle, h is the flight altitude, and b represents the airbrake deployment setting.

For the HYDDB dataset, the input feature space is bounded as reported in Table 3.

Table 3: Input feature ranges adopted for the HYDDB dataset.

Variable	Symbol	Range
Mach number	M	[0.3, 1.8]
Total angle of attack	α	[0°, 10°]
Roll angle	ϕ	[0°, 120°]
Altitude	h	[0 m, 10000 m]
Airbrake opening	b	[0%, 100%]

The dataset therefore consists of input-output pairs $(\mathbf{x}, C_A)$, where each sample corresponds to a single CFD simulation performed under specific flight conditions.

5.2 Dataset Typology

The dataset analysed in this work, **Hydra Database (HYDDB)**, is based on a fixed-geometry rocket, namely *Hydra*, the latest sounding rocket developed by the Skyward Experimental Rocketry student association. The geometry is held constant across all samples; the only design degree of freedom is the airbrake opening $b \in [0\%, 100\%]$, which combines with the flight conditions (Mach number, angle of attack, roll angle, altitude) to define each input vector $\mathbf{x}$. A description of the reference rocket geometry is provided in Appendix B.

Each entry of the dataset consists of $\mathbf{x}$ and the corresponding six aerodynamic coefficients $\mathbf{c}$ obtained from CFD simulations.

5.3 Automated CFD Pipeline

The generation of the datasets has been fully automated through a Python-based workflow leveraging the *pyAnsys* ecosystem. In particular, *pyFluent* has been used to interface with ANSYS Fluent and orchestrate the entire simulation pipeline.

For each sample, the following steps are executed automatically:

1. CAD generation using SpaceClaim, based on the fixed rocket geometry with the airbrake aperture b as the only varying design parameter;
2. mesh generation using Fluent Meshing, producing an unstructured polyhedral mesh;
3. setup and execution of a steady-state RANS simulation in Fluent.

This automated workflow enabled the large-scale generation of CFD data with minimal manual intervention. Additional details regarding mesh generation, boundary conditions, turbulence modeling, and solver settings are provided in Appendix D.

5.4 Sampling Strategy

A key challenge in dataset generation lies in the selection of the sampling points in the input space. Although the simulation pipeline is fully automated and leverages the native GPU solver available in Fluent, the computational cost of a single simulation remains significant, with an average runtime of approximately 400 seconds.

While this cost is relatively low compared to higher-fidelity approaches, the steady-state RANS simulations used here are still expensive enough to make dense, structured grids over a five-dimensional input space impractical. Therefore, an efficient sampling strategy is required to maximize the information content of the dataset.

To address this issue, an optimized *Latin Hypercube Sampling* (LHS) design has been adopted [6]. The starting point is a normalized feature space in which all inputs are first scaled to the unit hypercube $[0, 1]^5$. Within this space, the LHS construction enforces a stratified coverage of each dimension by partitioning it into N equally probable intervals and drawing one sample from each interval. The resulting one-dimensional strata are then combined across dimensions and optimized to improve the space-filling properties of the design, in the spirit of the sampling strategies advocated by Forrester et al.

Once the normalized sampling plan has been generated, the points are mapped back to the physical domain through variable-specific transformations. Let $\tilde{x}_j \in [0, 1]$ denote the normalized coordinate of the j-th dimension and let $[x_j^{\min}, x_j^{\max}]$ be the corresponding physical range. For the continuous variables, the denormalization uses the power-law mapping

$$x_j = x_j^{\min} + \tilde{x}_j^{p_j} \left(x_j^{\max} - x_j^{\min} \right),$$

with $p_j = 1$ for α, ϕ, and h, which reduces to a uniform linear rescaling over the physical interval. The airbrake opening is treated as a discrete variable: after the linear map to $[0\%, 100\%]$, the value is rounded to the nearest admissible opening level, i.e.

$$b = 5\% \cdot \text{round}\left(\frac{b^\star}{5\%} \right),$$

where $b^\star$ is the continuous value produced by the rescaling step. This enforces a sampling grid with steps of 5%. Only the Mach coordinate uses a centered power-law mapping with $p_M = 2$ and center $M_c = 1$, so that the design is locally concentrated around the transonic region. In practice, this is implemented by shifting the normalized coordinate with respect to the unit value corresponding to M_c and then applying the centered power law before the final physical rescaling.

This approach guarantees that:

- each input variable is well distributed across its range;
- clustering of points is avoided;

As a result, Latin Hypercube Sampling provides a good trade-off between coverage of the input space and number of required simulations, making it particularly suitable for the CFD-based dataset generation.

6 Results and Analysis on HYDDB

6.1 Evaluation Metrics

The surrogate models are evaluated using the coefficient of determination (R^2) and the Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2}, \tag{32}$$

The final results are reported as:

$$\mu \pm \sigma, \tag{33}$$

where μ and σ denote the mean and standard deviation computed across the 10 independent runs.

For the quantitative comparison, the split with 10% training data is omitted and the results are reported for the 20%, 50%, and 85% training configurations.

6.2 Quantitative Results

The architectures behind the numbers reported below are intentionally shallow: extensive hyperparameter exploration on HYDDB showed that oversized networks neither improve accuracy nor stabilise training, while smaller blocks remain easier to optimise jointly with the GP-side hyperparameters. In particular:

- **MLP**: a single hidden layer of width 128 with `tanh` activation, which proved sufficient to capture the aerodynamic response on this 5-dimensional input space.
- **MLP–GP$_{\mathbf{NS}}$**: a $32 \to 64 \to 32$ `tanh` feature extractor that predicts the local $\sigma(\mathbf{x})$ and $\ell(\mathbf{x})$ for the Gibbs kernel.
- **MLP–GP$_{\mathbf{DKL}}$**: the same $32 \to 64 \to 32$ `tanh` backbone projecting into a 3-dimensional feature space (output_width = 3) on which the exponential kernel is evaluated.

The GP baseline uses no neural component and is parameterised only by the kernel hyperparameters in eq. (9).

Tables 4, 5, and 6 summarize the predictive performance obtained for the different training set sizes.

Table 4: Predictive performance on HYDDB using 20% training data.

Model	$R^2 \uparrow$	**RMSE** $\downarrow$
MLP	0.986 ± 0.004	0.032 ± 0.005
GP	0.996 ± 0.001	0.016 ± 0.001
MLP–GP (NS)	0.995 ± 0.001	0.018 ± 0.001
MLP–GP (DKL)	0.988 ± 0.001	0.030 ± 0.007

Table 5: Predictive performance on HYDDB using 50% training data.

Model	$R^2 \uparrow$	**RMSE** $\downarrow$
MLP	0.996 ± 0.002	0.017 ± 0.004
GP	0.998 ± 0.000	0.012 ± 0.001
MLP–GP (NS)	0.998 ± 0.001	0.013 ± 0.002
MLP–GP (DKL)	0.997 ± 0.001	0.015 ± 0.002

Table 6: Predictive performance on HYDDB using 85% training data.

Model	$R^2 \uparrow$	**RMSE** $\downarrow$
MLP	0.998 ± 0.001	0.013 ± 0.003
GP	0.998 ± 0.000	0.011 ± 0.001
MLP–GP (NS)	0.999 ± 0.000	0.010 ± 0.001
MLP–GP (DKL)	0.999 ± 0.000	0.011 ± 0.001

6.3 Sampling-Plan Coverage

To make the space-filling properties of the HYDDB sampling plan explicit, Figure 1 reports the predicted C_A–Mach slices at $\alpha = 0°$, $\phi = 0°$, and $h = 5000\,\mathrm{m}$ for representative airbrake openings b, for all four surrogate models (MLP, GP, MLP–GP$_{\mathrm{NS}}$, and MLP–GP$_{\mathrm{DKL}}$). The shaded bands correspond to the $\pm 2\sigma$ predictive interval for the GP-family models; the MLP panel only reports the point prediction since it does not provide a posterior variance. The markers are only a projection of a subset of the samples onto this specific slice, so they should be read as a local view of the sampling plan rather than as the full cloud of CFD points.

6.4 Uncertainty Reliability

Figure 2 compares the empirical coverage on the test set against the nominal Gaussian coverage for the three GP-family surrogates (GP, MLP–GP$_{\mathrm{NS}}$, and MLP–GP$_{\mathrm{DKL}}$), using prediction bands of the form $\pm z\sigma$. A perfectly calibrated model follows the Gaussian reference curve. The GP curve is systematically conservative, indicating a clear overestimation of predictive uncertainty. The MLP–GP$_{\mathrm{NS}}$ curve is also slightly conservative but follows the Gaussian calibration curve much more closely. The MLP–GP$_{\mathrm{DKL}}$ curve drops below the reference at moderate band widths, hinting at a mild undercoverage that should be monitored when using it for confidence intervals. The plot is meant as a com-

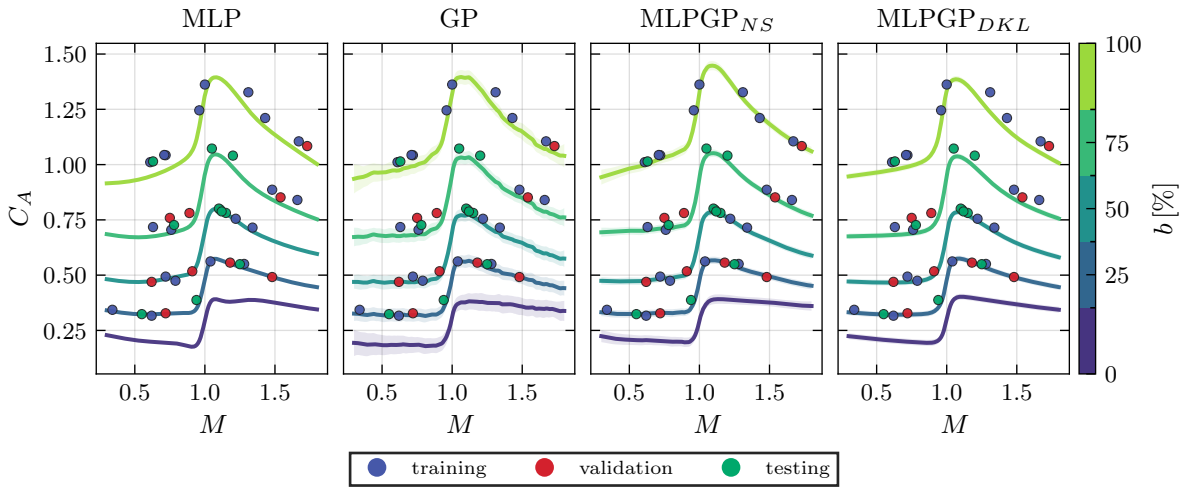


Figure 1: HYDDB sampling coverage on a representative C_A–Mach slice at $\alpha = 0°$, $\phi = 0°$, and $h = 5000$ m, for the four surrogate models. The MLP panel only reports the point prediction; the GP-family panels also report the $\pm 2\sigma$ band. The markers show only a projected subset of samples on this slice and are colored by train/validation/test split.

pact reliability check of the predictive uncertainty, using the test residuals standardized by the model-predicted standard deviation after denormalizing both predictions and targets back to the original output scale.

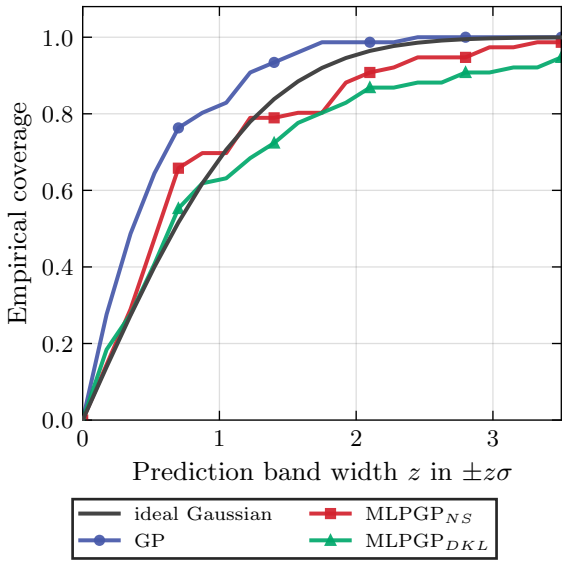


Figure 2: Uncertainty reliability on the HYDDB test set. The Gaussian reference curve is shown together with the empirical coverage of GP, MLP–GP$_{\text{NS}}$, and MLP–GP$_{\text{DKL}}$. The GP is consistently conservative; MLP–GP$_{\text{NS}}$ tracks the Gaussian calibration curve much more closely; MLP–GP$_{\text{DKL}}$ sits below the reference at moderate band widths, indicating mild under-coverage.

6.5 Discussion

The obtained results highlight the different behaviors of the surrogate models across varying data regimes.

In the low-data configuration, Gaussian Processes generally benefit from their strong inductive bias and intrinsic regularization properties, often providing robust predictions even with limited training samples. Conversely, purely neural approaches tend to require larger datasets to fully exploit their representational capacity.

As the amount of training data increases, MLP-based models progressively improve their approximation capability, particularly in the presence of highly nonlinear aerodynamic interactions. The hybrid MLP–GP approaches provide an intermediate compromise, combining the flexibility of learned feature representations with the probabilistic structure of kernel regression.

The standard deviation associated with the reported metrics also provides insight into the robustness of the optimization process and the sensitivity of each model to initialization and dataset partitioning.

7 Conclusions and Future Work

7.1 Conclusions

This work compared a purely neural surrogate (MLP), an exact Gaussian Process with a learnable power–exponential kernel (GP), and two hybrid MLP–GP variants (non-stationary Gibbs kernel and deep kernel learning) on the HYDDB axial-coefficient prediction task across different data regimes.

The hybrid MLP–GP architectures examined here can yield a measurable benefit over a plain GP or a plain MLP, but this benefit is clearly dataset-dependent. On HYDDB the GP baseline is already very strong in the low-data regime, where its inductive bias dominates; the hybrid variants close the gap only as the training set grows and the non-stationary or learned-feature representation starts to pay off. The improvement on a different problem– one with stronger non-stationarity in the input space, or higher input dimensionality– could be larger, but is not something that can be claimed in general from a single benchmark.

The increased modelling capacity of MLP–GP also comes with a non-trivial cost that should be acknowledged when choosing the surrogate family:

- **Training and inference time.** Each forward pass goes through both an MLP feature extractor and a full kernel evaluation; backpropagating through the Cholesky factorization of the NLL (23) is more expensive than a plain MLP backward pass, and slower than the closed-form GP gradient. Inference is also heavier than a pure MLP because the kernel matrix has to be re-evaluated against the training set.
- **Optimization landscape.** Jointly training the MLP weights $\boldsymbol{\theta}_{\text{MLP}}$ and the kernel hyperparameters $\boldsymbol{\theta}_k$ creates a markedly non-convex objective with strongly coupled parameter groups, which motivated the separate learning rates η_{MLP} and η_{GP} adopted in this work (Tab. 1).
- **Overfitting and ill-conditioning.** The MLP can collapse the learned features into degenerate configurations that leave the kernel matrix near-singular, which combined with low likelihood noise leads to ill-conditioned Cholesky solves. In practice this has to be controlled through jitter, bounded likelihood noise, and weight decay on the MLP block.

Under appropriate conditions these costs are repaid by higher accuracy and a more faithful reconstruction of the underlying response surface than either of the two parents alone. In particular, the DKL formulation projects the inputs into a lower-dimensional feature space on which the kernel matrix is evaluated, which is also a natural entry point for improving GP scalability when combined with non-exact GP formulations (e.g. sparse / variational approximations) that were intentionally left out of the present study.

7.2 Future Work

A number of extensions are natural directions from the work presented here:

- **Variable geometry.** Re-introducing geometric design parameters (boat tail, fin chord, nose profile, etc.) on top of the five flight inputs would push the surrogate into a substantially higher-dimensional space, where the trade-off between GP and hybrid MLP–GP is expected to shift in favour of learned features.
- **Mixture of experts.** The four canonical models could be combined in a gated mixture in which an MLP selects, per region of the input space, the most appropriate specialist (e.g. pure GP in low-data regions, MLP–GP_{DKL} in regions with strong non-stationarity), trading complexity for local accuracy.
- **Physics-informed extensions.** Embedding physical constraints–symmetries, monotonicity, simple equilibrium relations among aerodynamic coefficients–into the loss or the kernel is particularly attractive in the multi-output setting, where the six components of $\mathbf{c}$ are not independent and sharing structure across them can both regularise training and improve predictions on the rarer coefficients.
- **Multi-output formulation.** Extending the present framework from the single coefficient C_A to the full vector $\mathbf{c} = [C_A, C_Y, C_N, C_l, C_m, C_n]^T$ via multi-task GPs or coregionalisation kernels is a direct next step, and would benefit most from the physics-informed regularisation mentioned above.
- **Scalable GP inference.** Coupling the DKL feature extractor with sparse / variational GP inference would lift the cubic-cost limitation of exact training and make the framework applicable to substantially larger CFD datasets.

A ISA and Reynolds Number

In fundamental aerodynamics, the Reynolds number (Re) is the natural parameter for viscous scaling. However, in this project, geometric altitude (h) is retained at the interface level because it is directly

available as a state variable in standard 6-DOF trajectory simulation workflows.

Given the Mach number M and altitude h, the local Reynolds number can be deterministically reconstructed assuming the International Standard Atmosphere (ISA) model. Below the tropopause ($h < 11\,000$ m), the atmospheric temperature T and pressure p are computed as:

$$T(h) = T_0 - L \cdot h, \quad p(h) = p_0 \left(\frac{T(h)}{T_0} \right)^{\frac{g}{LR}},$$

where $T_0 = 288.15$ K and $p_0 = 101\,325$ Pa are the sea-level standard conditions, $L = 0.0065$ K/m is the temperature lapse rate, $g = 9.80665$ m/s^2 is the gravitational acceleration, and $R = 287.05$ J/(kg·K) is the specific gas constant for air. The local air density ρ follows from the ideal gas law, $\rho(h) = p(h)/(RT(h))$.

To accurately model the dynamic viscosity μ as a function of temperature, we employ Sutherland's law:

$$\mu(T) = \mu_0 \left(\frac{T}{T_0} \right)^{3/2} \frac{T_0 + S}{T + S},$$

where $\mu_0 = 1.8325 \times 10^{-5}$ kg/(m·s) is the reference viscosity at T_0, and $S = 110.4$ K is Sutherland's effective temperature constant for air.

Finally, the speed of sound is $a(h) = \sqrt{\gamma RT(h)}$ (with $\gamma = 1.4$), yielding the freestream velocity $V = M \cdot a(h)$. The Reynolds number is thus recovered as:

$$Re = \frac{\rho(h)\, V(M,h)\, L_{\text{ref}}}{\mu(T(h))},$$

where L_{ref} is the characteristic reference length of the rocket. This formulation guarantees that the multi-fidelity surrogate remains inherently tied to the correct viscous scaling, while exposing a practical and simulation-friendly (M, h) interface to the user.

B Reference Rocket Geometry

This appendix describes the geometric parameterization adopted for the rocket configurations used in the dataset generation. The parameterization is designed to allow controlled variations of the main aerodynamic features while maintaining consistency with a realistic sounding rocket architecture. For reference, the nominal values corresponding to the Hydra configuration, used in the HYDDB dataset, are explicitly reported throughout this appendix.

B.1 Global Rocket Geometry

The main body of the rocket is defined through a set of parameters controlling the overall dimensions, the nose profile, and the aft boat-tail geometry.

Table 7: Nominal global geometry parameters for the Hydra configuration.

Parameter	**Value**
Rocket diameter (D)	180 mm
Total length (L_{tot})	3078 mm
Nose length (L_{nose})	440 mm
Nose exponent (P)	1.288
Haack coefficient (C)	0.254
Boat-tail length (L_{boat})	480 mm
Boat-tail exit diameter (D_{boat})	81 mm

The nose cone is modeled using a modified Haack-series parameterization. The profile is generated by introducing an additional exponent parameter P into the classical Haack formulation, allowing greater flexibility in the curvature distribution and enabling deviations from standard LV-Haack or von Kármán profiles.

The nose radial coordinate is defined as:

$$r(x) = \frac{R}{\sqrt{\pi}} \sqrt{\theta - \frac{1}{2}\sin(2\theta) + C\sin^3(\theta)} \tag{34}$$

with

$$\theta = \arccos\left(1 - 2\frac{x_m}{L_{\text{nose}}}\right) \tag{35}$$

and

$$x_m = \left(\frac{x}{L_{\text{nose}}}\right)^P L_{\text{nose}}. \tag{36}$$

where:

- $R = D/2$ is the rocket radius;
- L_{nose} is the nose length;
- C is the Haack shape coefficient;

- P is an additional shaping exponent controlling the axial redistribution of curvature.

This modified formulation allows smooth transitions between sharper and fuller nose geometries while preserving a continuously differentiable profile.

The aft section includes a boat-tail geometry generated using a cubic Bézier curve parameterization:

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \qquad t \in [0,1]. \tag{37}$$

The control points are defined as:

$$\mathbf{P}_0 = (x_0, R), \tag{38}$$

$$\mathbf{P}_1 = (x_0 - x_1, R), \tag{39}$$

$$\mathbf{P}_2 = (x_0 - x_2, y_2), \tag{40}$$

$$\mathbf{P}_3 = (x_0 - L_{\text{boat}}, R_b), \tag{41}$$

where

$$R_b = \frac{D_{\text{boat}}}{2}. \tag{42}$$

Table 8: Boat-tail Bézier control parameters.

Parameter	Value
First axial offset (x_1)	225 mm
Second axial offset (x_2)	451 mm
Intermediate radial coordinate (y_2)	58 mm

This parameterization guarantees a smooth and controllable contraction of the aft body while maintaining geometric continuity with the cylindrical fuselage.

A simplified flat-base assumption is adopted for the rocket aft face. The nozzle geometry and the internal cavities located in the base region are not modeled explicitly. This simplification was introduced because, during the development of the present work, the propulsion integration and thermal design of the aft section were still subject to significant modifications. The adopted approximation ensures geometric consistency across the dataset while avoiding the use of non-representative nozzle configurations.

B.2 Fin Geometry

The stabilizing fins are defined through a trapezoidal planform parameterization with finite thickness.

Table 9: Nominal fin geometry parameters.

Parameter	Value
Root chord (c_{root})	400 mm
Tip chord (c_{tip})	119 mm
Fin height (h_{fin})	180 mm
Chord offset (Δx)	270 mm
Axial position from tube end	10 mm
Semi-thickness ($t/2$)	1.2 mm
Leading-edge chamfer angle	45°
Trailing-edge chamfer angle	45°

The parameter Δx controls the fin sweep by defining the axial displacement between the root and tip chords.

Both leading and trailing edges are modeled using symmetric 45° chamfers to reduce sharp-edge numerical artifacts and provide a more realistic representation of the manufactured fins.

B.3 Airbrake System

The airbrake (ABK) system is modeled as a deployable aerodynamic surface integrated into the rocket body. Its geometry is defined through both fixed shape parameters and a deployment variable.

Table 10: Nominal airbrake geometry parameters.

Parameter	Value
Opening percentage (b)	Variable
Theoretical deployment angle	110.38°
Effective deployment angle	120°
Maximum extraction	54 mm
Airbrake width	6 mm
Axial position from aft reference	1167 mm

The opening percentage parameter b controls the deployment state of the airbrakes and is defined as the percentage of the maximum deployable aerodynamic area. Consequently, a deployment value

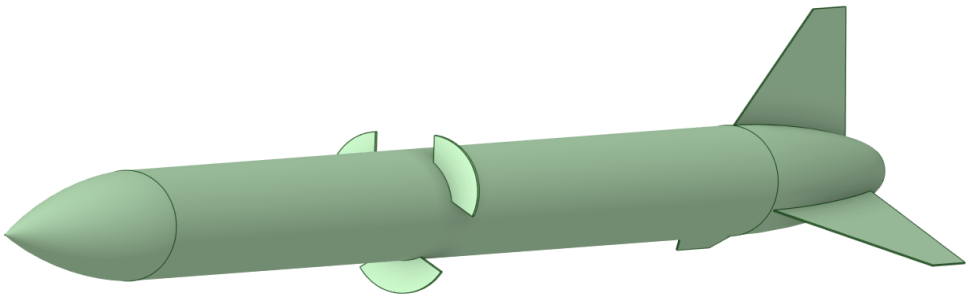


Figure 3: Complete Hydra rocket geometry with fully deployed airbrakes. The figure highlights the main aerodynamic components, including the modified Haack nose profile, cylindrical body, boat-tail, fin can, and airbrake system.

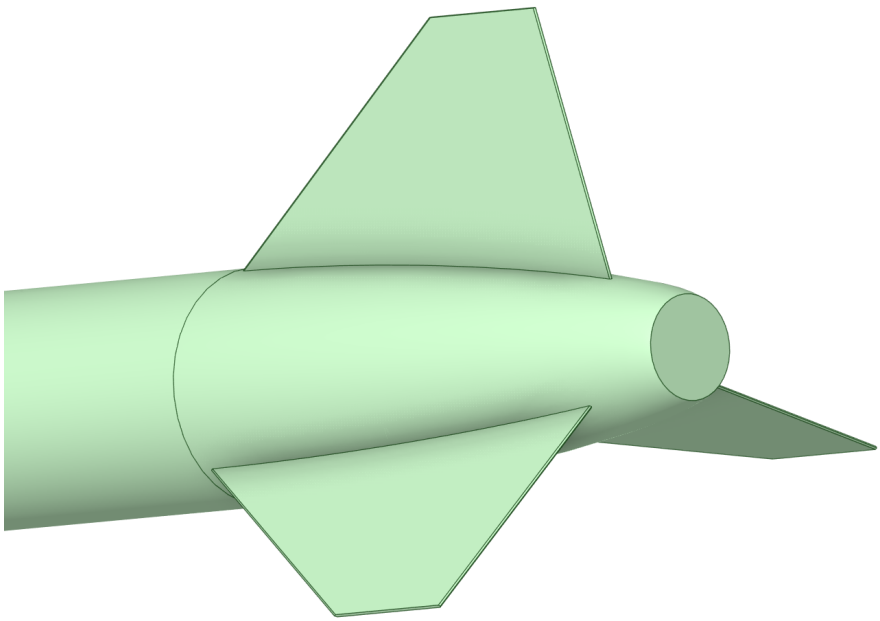


Figure 4: Detailed view of the fin can region, including the trapezoidal fin planform, boat-tail geometry, and edge chamfers.

of 50% corresponds to half of the maximum exposed petal area, while 100% represents the fully deployed configuration.

This parameter plays a key role in modulating the drag of the rocket across the simulated flight envelope.

B.4 Use within HYDDB

The Hydra Database (HYDDB) adopts the nominal Hydra geometry described in this appendix as a fixed reference, varying only the flight conditions and the airbrake deployment b across samples. The remaining parameterization is documented here for reference and to support future extensions of the dataset.

C Mesh Generation

This appendix describes the computational domain and mesh generation strategy adopted for the CFD simulations used to populate the aerodynamic database.

C.1 Computational Domain

The fluid domain is designed to accurately reproduce external aerodynamic conditions while minimizing any interference induced by the numerical boundaries.

The domain consists of two main regions:

- a **hemispherical region** centered at the rocket nose tip;
- a **cylindrical extension** aligned with the rocket longitudinal axis.

The characteristic dimensions of the domain are defined as multiples of the total rocket length L_{tot}:

- **Hemisphere diameter:** $8L_{\text{tot}}$
- **Cylinder diameter:** $8L_{\text{tot}}$
- **Cylinder length:** $12L_{\text{tot}}$

This configuration ensures that the far-field boundaries are placed sufficiently far from the rocket to avoid spurious reflections, confinement effects, and artificial interactions with shock structures and wake development.

C.2 Mesh Generation Strategy

The computational mesh is generated using *ANSYS Fluent Meshing 2025 R2*. The fluid domain is discretized using **polyhedral cells**, which provide an effective compromise between numerical accuracy and computational cost.

The main drawback of polyhedral discretization is a slightly higher numerical diffusion, which is mitigated through the use of *warped face gradient correction* within the solver.

C.3 Boundary Layer Resolution

To accurately resolve near-wall gradients, prism inflation layers are generated over rocket surfaces.

The mesh is designed to satisfy:

$$y^+ < 5$$

throughout the entire flight envelope, allowing proper integration of the governing equations within the assumptions of the selected turbulence model.

Special attention is devoted to regions characterized by strong geometric curvature and flow separation, including:

- the nose cone;
- the airbrake petals;
- the boat-tail;
- the fin can.

In these regions, local mesh refinement is applied both on the surface and within the surrounding volume to accurately capture shock waves, separated flow structures, and wake interactions.

C.4 Surface Mesh Details

Figures 8–10 show representative close-up views of the surface mesh in the most critical regions of the geometry.

C.5 Mesh Size and Case-to-Case Variability

The total number of cells depends on the specific geometry being analyzed. Variations in geometric parameters, such as airbrake deployment or modifications to the aerodynamic surfaces, require:

- regeneration of the CAD model;

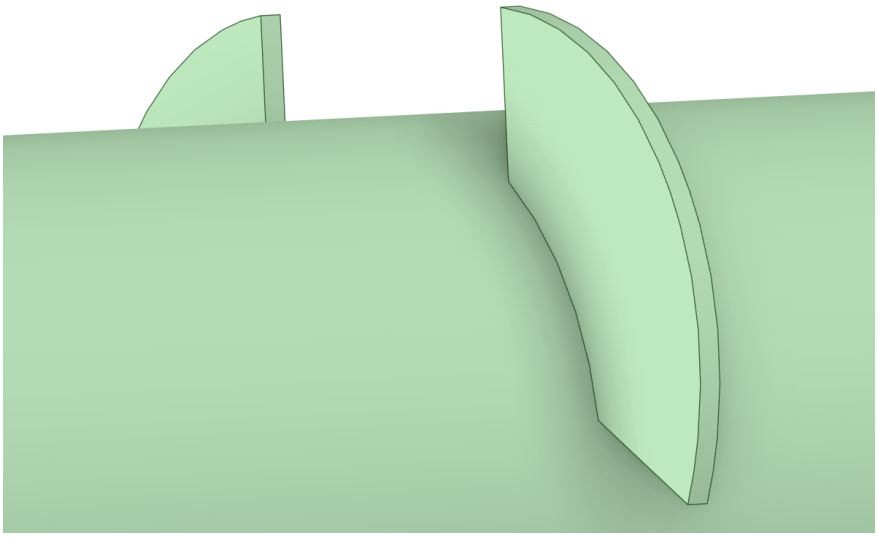


Figure 5: Detailed view of the airbrake system in the fully deployed configuration, showing the deployment angle definition and extraction geometry.

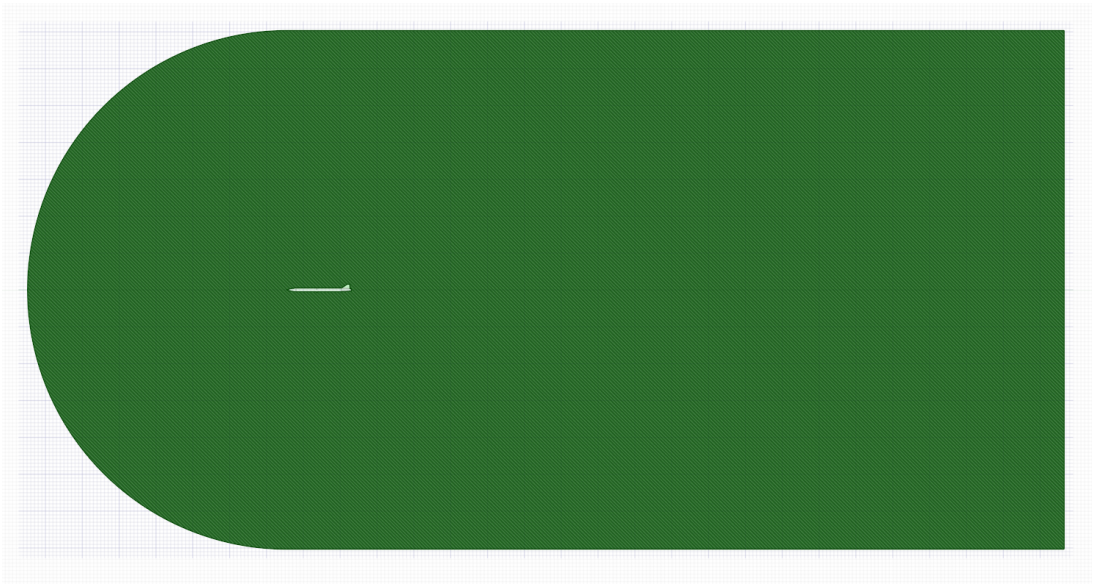


Figure 6: Computational fluid domain composed of a hemispherical upstream region and a cylindrical downstream extension.

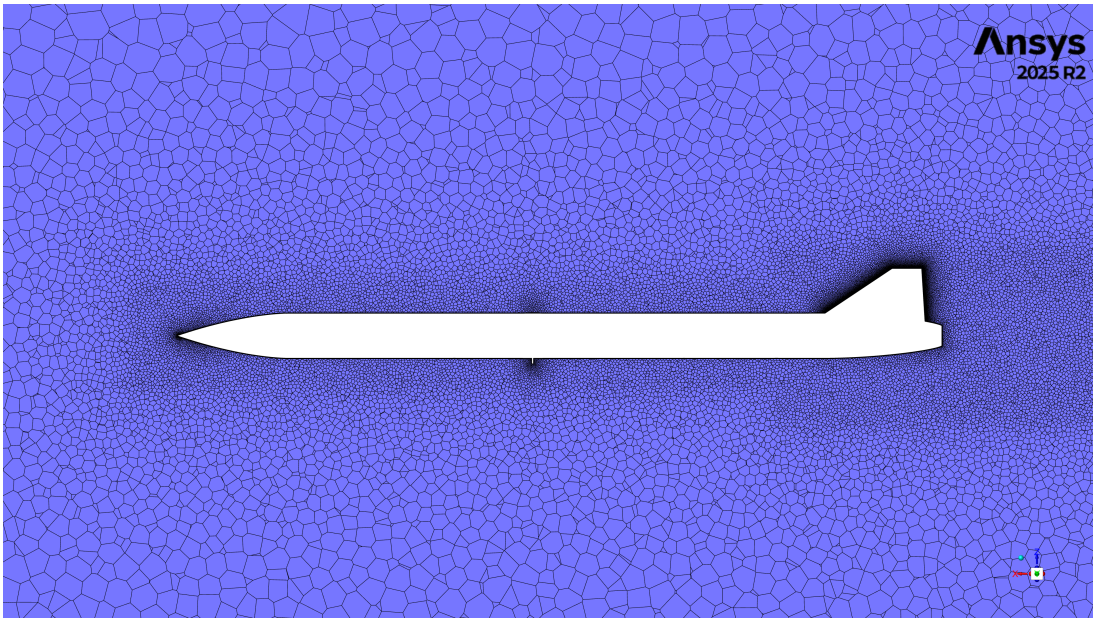


Figure 7: Section of the computational mesh in the x–z plane, highlighting the polyhedral volume discretization.

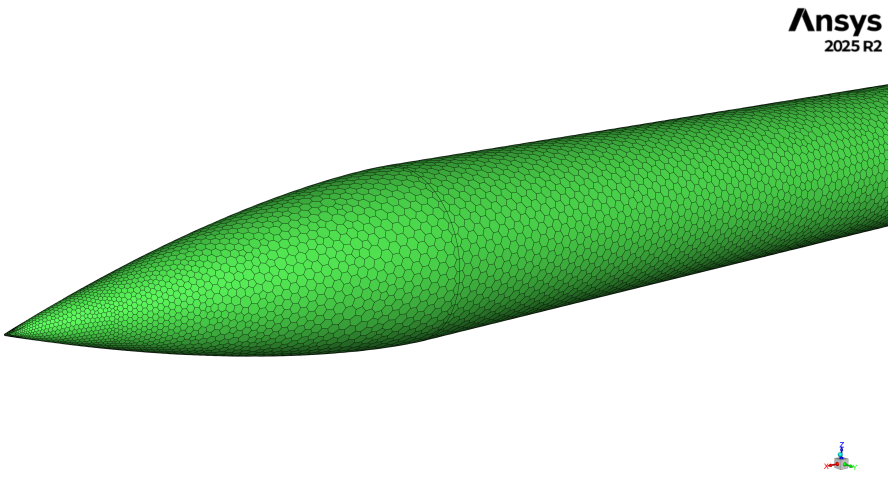


Figure 8: Surface mesh detail on the modified Haack nose cone.

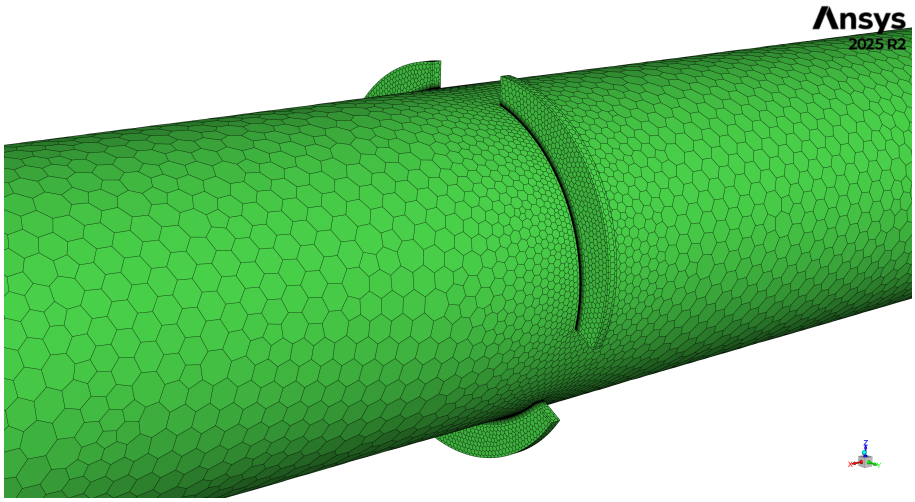


Figure 9: Surface mesh detail in the airbrake region, showing local refinement around the deployed petals.

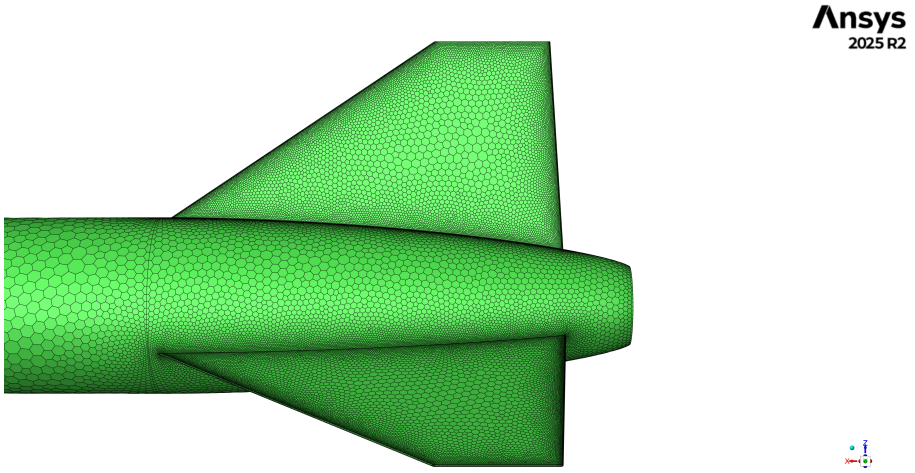


Figure 10: Surface mesh detail in the fin can region, including the boat-tail and stabilizing fins.

- regeneration of the computational mesh.

As a result, the mesh topology and total cell count are not fixed across the database.

For the majority of the simulations, the final mesh size is approximately:

$$\mathcal{O}(4 \times 10^6) \text{ cells}$$

with moderate variations depending on the complexity of the deployed airbrake configuration and the extent of local refinements.

This relatively limited cell count is made possible by the efficiency of polyhedral discretization, which requires significantly fewer cells than tetrahedral-based approaches to achieve comparable levels of accuracy.

D CFD simulation setup

This appendix summarizes the main numerical settings adopted for the CFD simulations used to generate the aerodynamic datasets. Further details on the numerical methods and implementation can be found in the ANSYS Fluent 2025 R2 documentation, namely the Theory Guide and the User's Guide [2, 3].

D.1 Solver Configuration

All simulations were performed using the ANSYS Fluent 2025 R2 GPU solver in single precision, executed on an NVIDIA RTX 4080 SUPER GPU. This configuration provides an effective compromise between computational efficiency and solution accuracy, making it particularly suitable for large-scale parametric studies involving thousands of operating conditions.

A pressure-based coupled solver was adopted, combined with pseudo-time stepping to accelerate convergence toward steady-state conditions. The simulations were conducted assuming fully compressible flow, with the energy equation enabled to properly capture thermodynamic effects over the entire flight envelope.

D.2 Turbulence Modeling

The flow field was modeled using the steady-state Reynolds-Averaged Navier–Stokes (RANS) equations. Turbulence effects were captured using the k–ω SST (Shear Stress Transport) model, which combines the near-wall accuracy of the k–ω formulation with the robustness of the k–ϵ model in the freestream.

This model is widely recognized as a reliable choice for external aerodynamic simulations involving adverse pressure gradients, shock-boundary layer interactions, and potential separation phenomena.

Viscous heating effects were included, which is particularly important for the transonic and supersonic flow regimes encountered during ascent.

D.3 Fluid Properties

The working fluid was air, modeled as an ideal gas. The thermophysical properties were defined as follows:

- **Density:** computed using the ideal gas law;
- **Dynamic viscosity:** evaluated using Sutherland's law;
- **Specific heat at constant pressure (c_p):** defined through Fluent's piecewise polynomial formulation;
- **Thermal conductivity:** assumed constant.

This formulation ensures an accurate representation of compressibility effects and temperature-dependent transport properties.

D.4 Boundary Conditions

The boundary conditions were defined to reproduce external aerodynamic conditions representative of free-flight operation:

- **Rocket surface:** modeled as a no-slip, adiabatic wall;
- **Far-field boundaries:** modeled using Fluent's *pressure-far-field* condition, where freestream Mach number, static pressure, and static temperature were specified according to the selected flight condition.

These settings provide a physically consistent simulation environment over all sampled combinations of Mach number, altitude, angle of attack, roll angle, and airbrake deployment.

D.5 Representative Flow Field Visualization

Figure 11 shows a representative contour of the pressure coefficient distribution obtained on the

Hydra rocket configuration.

The selected case corresponds to the following operating condition:

- Mach number: $M = 1.530$;
- Altitude: $h = 4128$ m;
- Total angle of attack: $\alpha = 8.0°$;
- Roll angle: $\phi = 109.20°$;
- Airbrake deployment: $b = 50\%$.

The contour highlights the complex flow structures captured by the simulations, including bow shocks, localized compression regions, expansion zones, and the strong pressure gradients generated around the partially deployed airbrakes. The asymmetry introduced by the nonzero angle of attack and roll angle is also clearly visible.

This qualitative visualization complements the quantitative aerodynamic database and provides additional confidence in the capability of the adopted numerical methodology to accurately capture the relevant physical phenomena governing the aerodynamic response of the vehicle.

E CFD simulation setup validation

This appendix summarizes the validation campaign carried out to assess the reliability of the CFD setup adopted for the generation of the aerodynamic datasets. The objective of this activity was to verify that the numerical methodology described in Appendix D is capable of accurately reproducing experimentally measured aerodynamic coefficients over a representative range of subsonic flow conditions.

The validation was performed by comparing ANSYS Fluent simulations against published wind tunnel measurements for the AGARD Model B calibration geometry, a widely used benchmark in aerodynamic analysis. The reference experimental dataset is taken from Anderson [1].

E.1 Validation Benchmark

The AGARD Model B is a canonical body-wing configuration extensively used for the calibration of wind tunnel facilities and for CFD code validation. Its geometry consists of a cylindrical fuselage with a tangent ogive nose and a delta wing of equilateral triangular planform.

A comprehensive experimental database is available for Mach numbers ranging from 0.2 to 1.0 and angles of attack between approximately $-4°$ and $24°$, making this model particularly suitable for assessing the predictive capabilities of aerodynamic simulation tools.

E.2 Numerical Setup

The numerical setup employed for this validation is fully consistent with the one used for the aerodynamic database generation, with only minor modifications required to reproduce the AGARD Model B geometry and the corresponding experimental conditions.

The main simulation settings are:

- Pressure-based coupled solver;
- Steady-state Reynolds-Averaged Navier–Stokes (RANS) equations;
- k–ω SST turbulence model;
- Ideal gas model for air;
- Second-order spatial discretization;
- Boundary-layer inflation with target $y^+ \approx 1$;
- Convergence monitored through both residuals and aerodynamic coefficients.

The computational domain was chosen sufficiently large to minimize blockage and boundary effects. Local mesh refinements were introduced in the wing and base regions to ensure adequate resolution of the flow features governing the aerodynamic loads.

E.3 Validation Matrix

The validation campaign considered the compressible CFD setup over a range of subsonic Mach numbers representative of the operating conditions relevant to the present work.

The following Mach numbers were analyzed:

- $M = 0.3$;
- $M = 0.5$;
- $M = 0.8$;
- $M = 1.0$.

For each case, aerodynamic coefficients were computed over a range of angles of attack and compared against the corresponding experimental measurements.

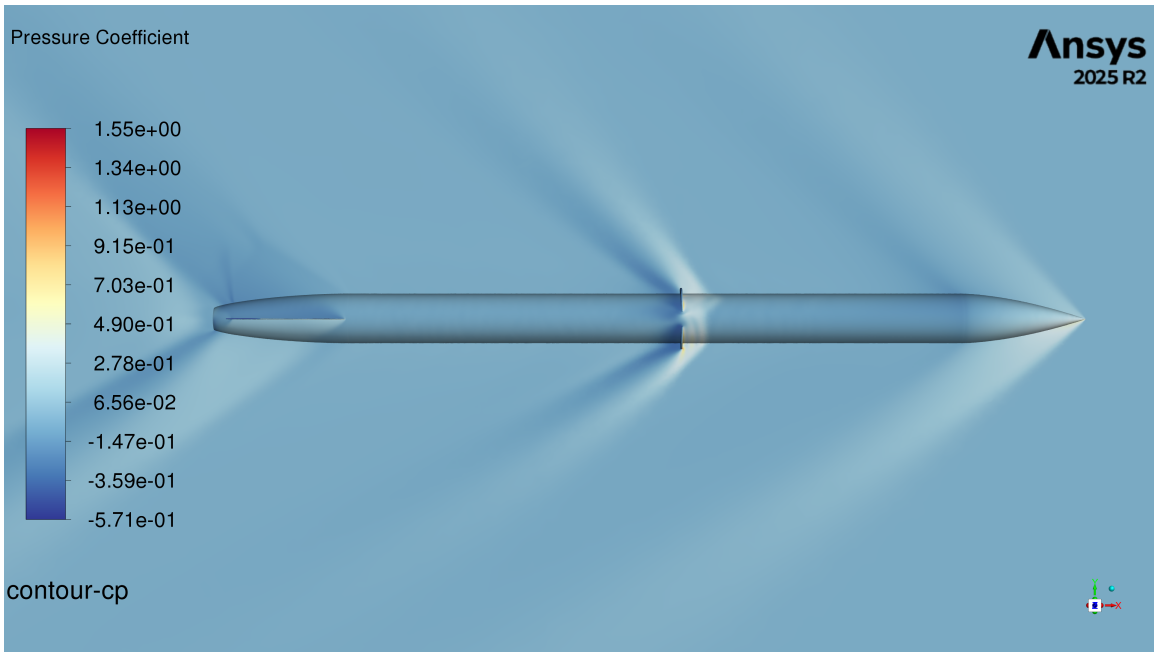


Figure 11: Pressure coefficient contour on the Hydra rocket for a representative transonic/supersonic operating condition ($M = 1.530$, $h = 4128$ m, $\alpha = 8.0°$, $\phi = 109.20°$, and airbrake deployment $b = 50\%$).

E.4 Validation Metrics

The comparison was performed in terms of the following aerodynamic coefficients:

- Drag coefficient, C_D;
- Lift coefficient, C_L;
- Pitching moment coefficient, C_M.

Together, these quantities provide a comprehensive assessment of the ability of the CFD model to predict both force and moment characteristics.

E.5 Results at $M = 0.3$

Figures 12–14 compare the numerical results with the wind tunnel measurements at Mach 0.3.

The numerical predictions are in excellent agreement with the experimental data, confirming the capability of the solver to accurately reproduce low-subsonic aerodynamic behavior.

E.6 Results at $M = 0.5$

Figures 15–17 report the comparisons at Mach 0.5.

Excellent agreement is observed for all aerodynamic coefficients, with the CFD solution accurately capturing both the slope and nonlinear behavior of the experimental curves.

E.7 Results at $M = 0.8$

Figures 18–20 show the results obtained at Mach 0.8.

The CFD model reproduces the experimental trends with very good accuracy, including the compressibility effects that become increasingly relevant in this regime.

E.8 Results at $M = 1.0$

Figures 21–23 present the comparison at Mach 1.0.

The transonic case also shows good agreement with the experimental data, demonstrating that the adopted numerical setup remains robust and accurate up to Mach 1.

E.9 Overall Assessment

Across all investigated Mach numbers, the CFD simulations reproduce the experimental aerodynamic coefficients with a high level of accuracy. The main observations are summarized below:

- Lift and pitching moment coefficients are predicted very accurately over the entire angle-of-attack range.
- Drag predictions show slightly larger discrepancies, which remain within typical engineering tolerances.

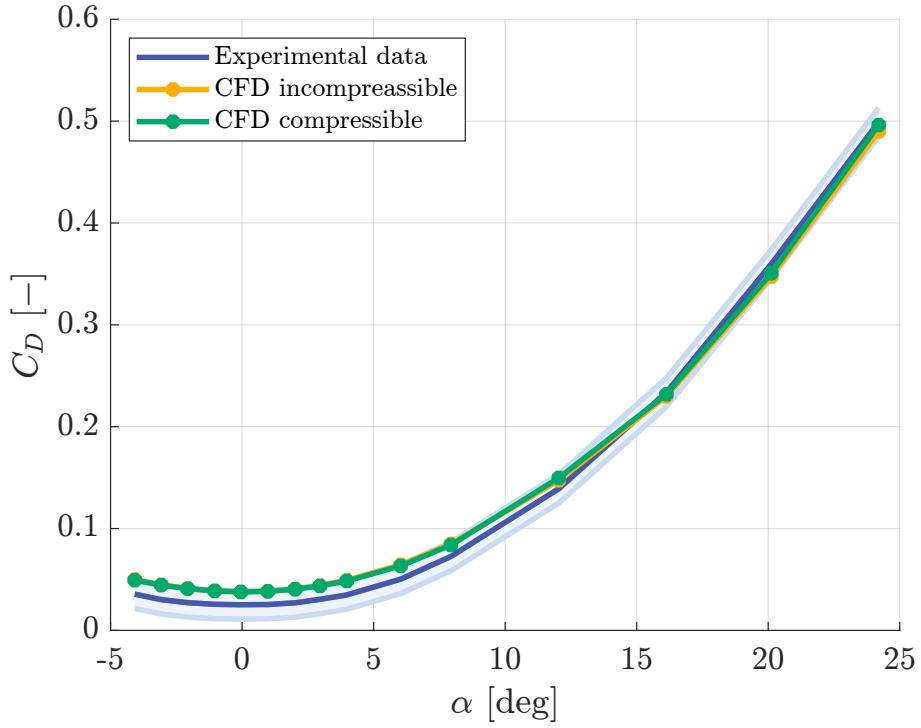


Figure 12: Comparison of drag coefficient C_D versus angle of attack at $M = 0.3$.

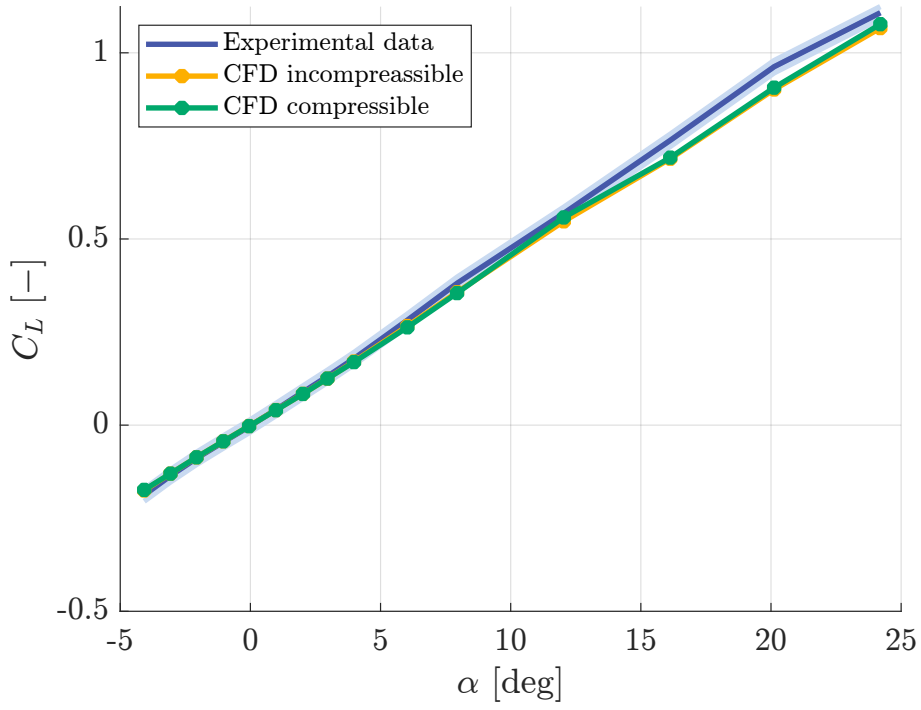


Figure 13: Comparison of lift coefficient C_L versus angle of attack at $M = 0.3$.

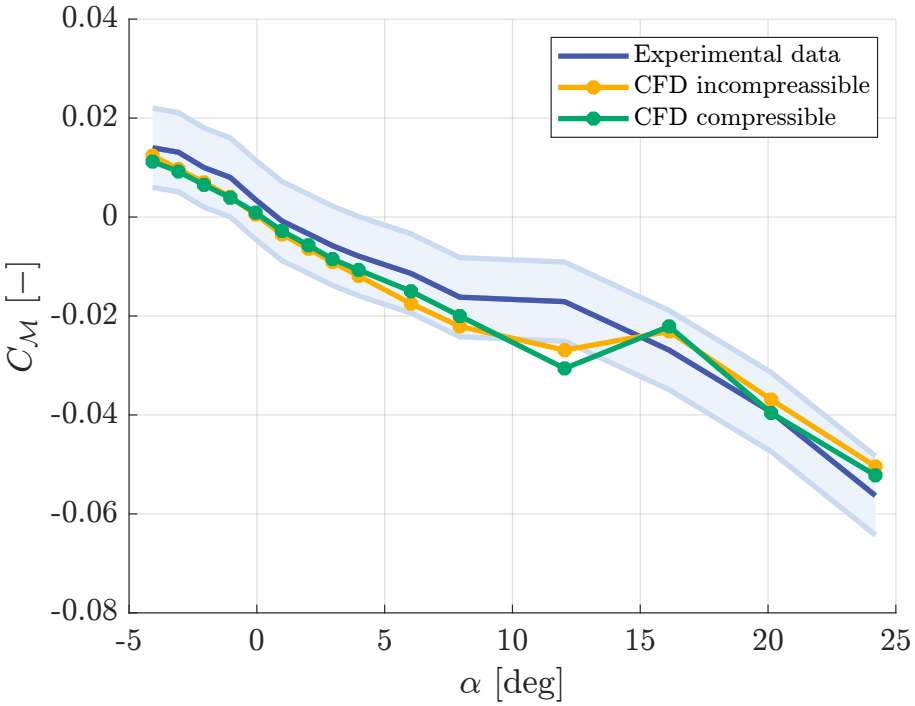


Figure 14: Comparison of pitching moment coefficient C_M versus angle of attack at $M = 0.3$.

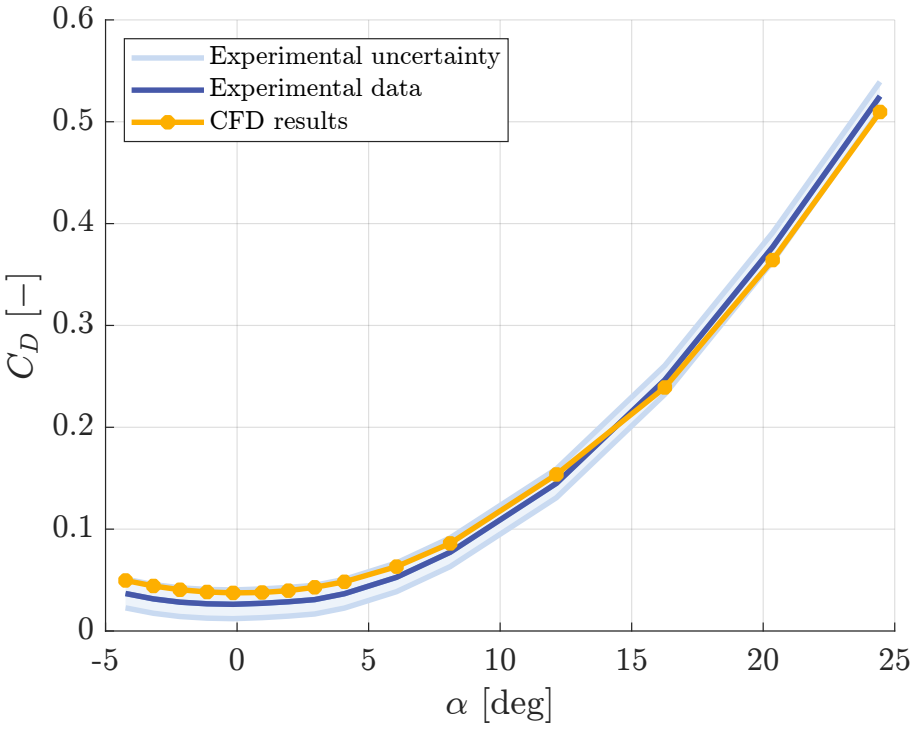


Figure 15: Comparison of drag coefficient C_D versus angle of attack at $M = 0.5$.

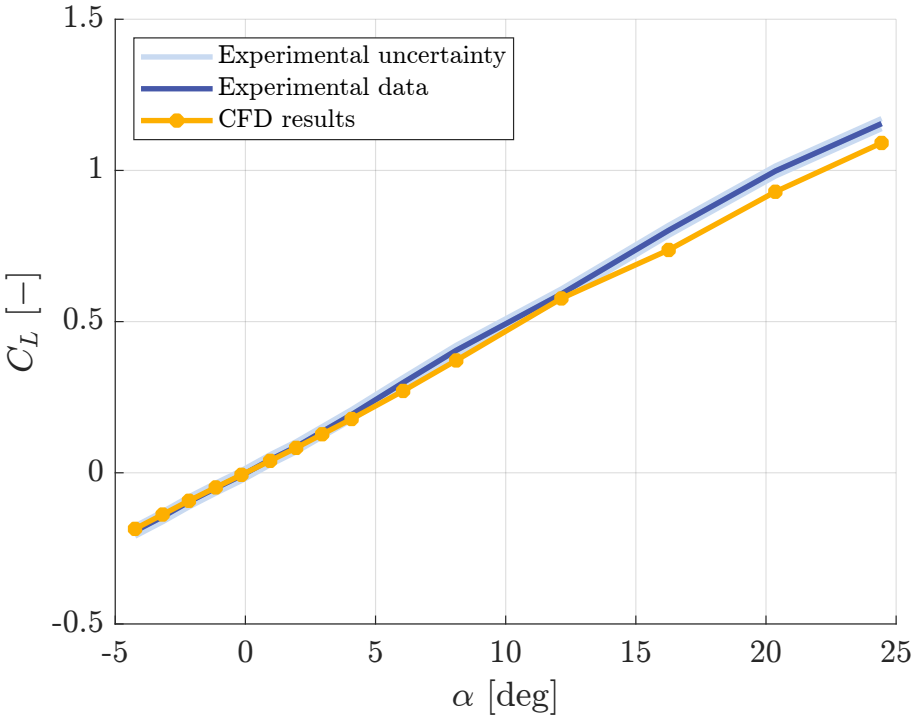


Figure 16: Comparison of lift coefficient C_L versus angle of attack at $M = 0.5$.

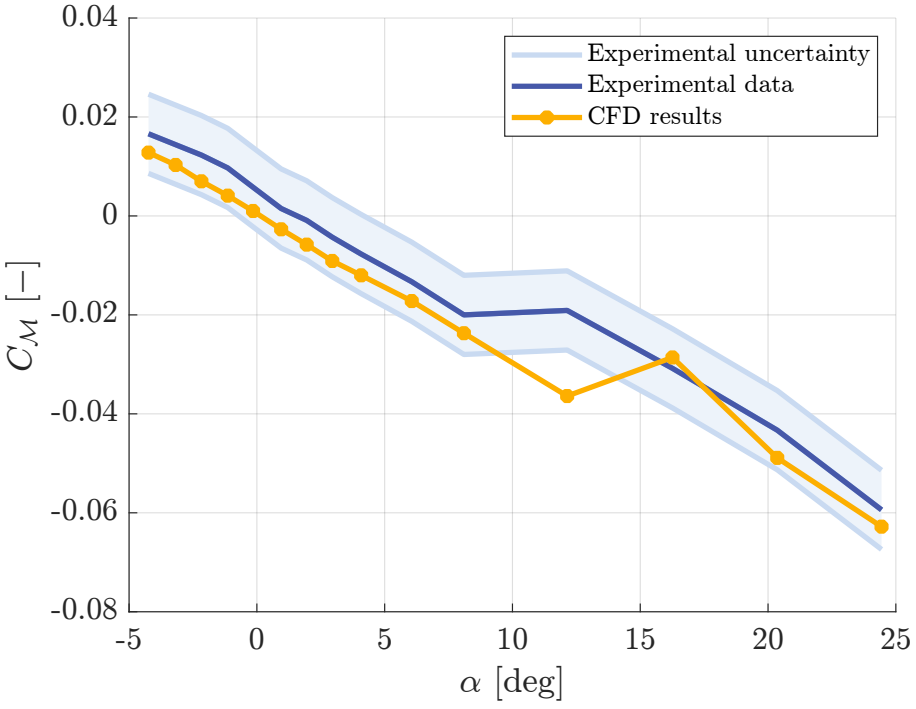


Figure 17: Comparison of pitching moment coefficient C_M versus angle of attack at $M = 0.5$.

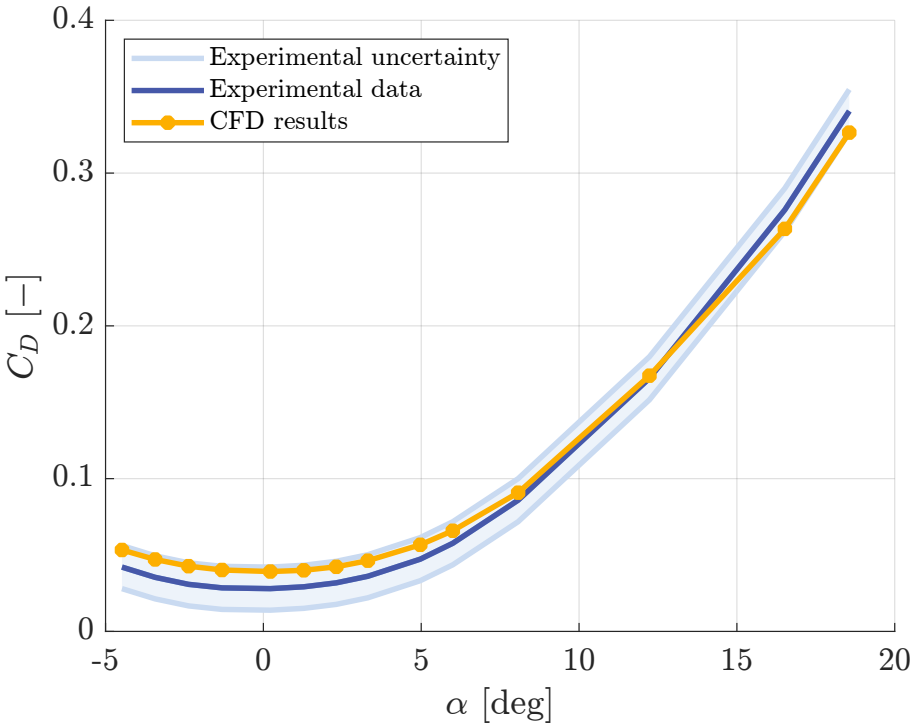


Figure 18: Comparison of drag coefficient C_D versus angle of attack at $M = 0.8$.

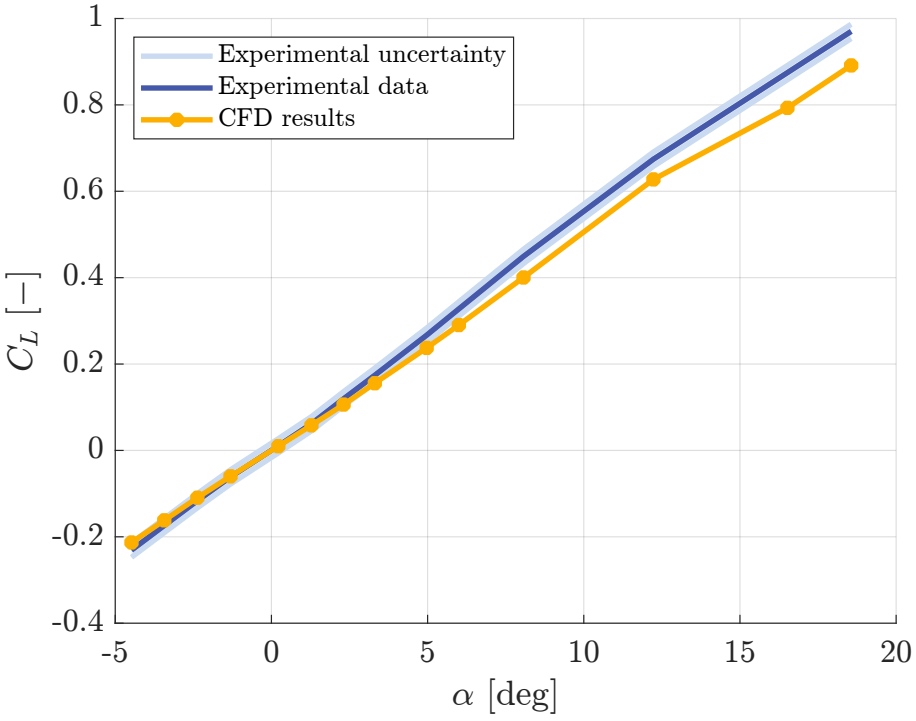


Figure 19: Comparison of lift coefficient C_L versus angle of attack at $M = 0.8$.

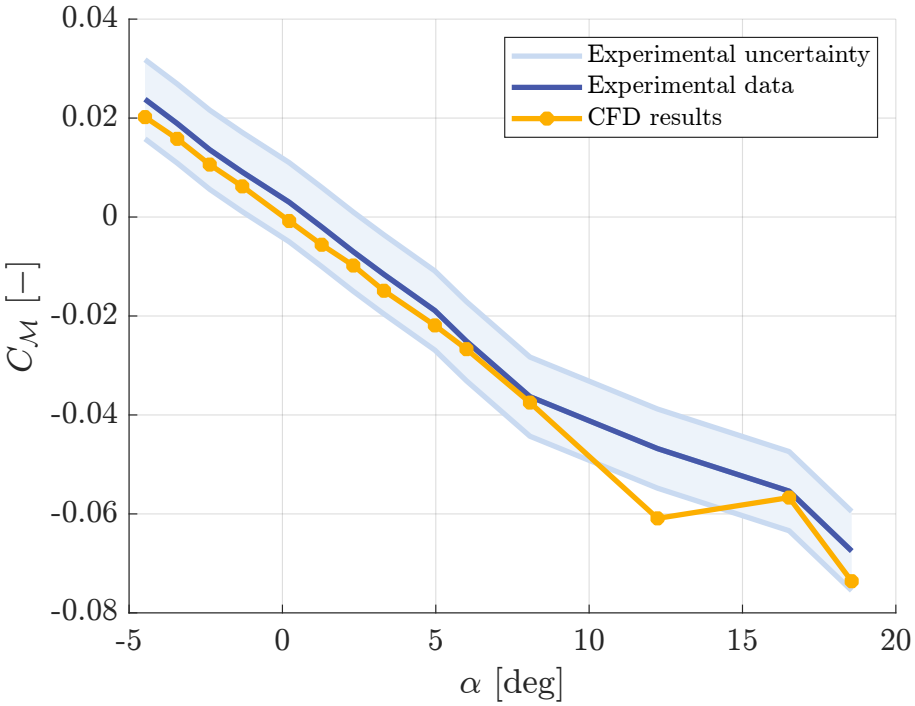


Figure 20: Comparison of pitching moment coefficient C_M versus angle of attack at $M = 0.8$.

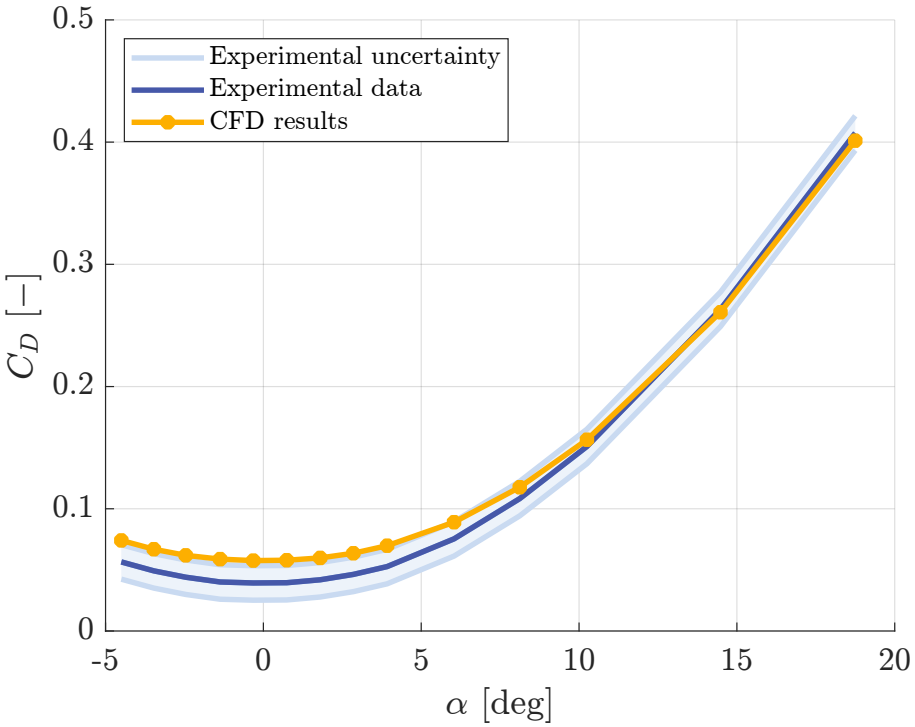


Figure 21: Comparison of drag coefficient C_D versus angle of attack at $M = 1.0$.

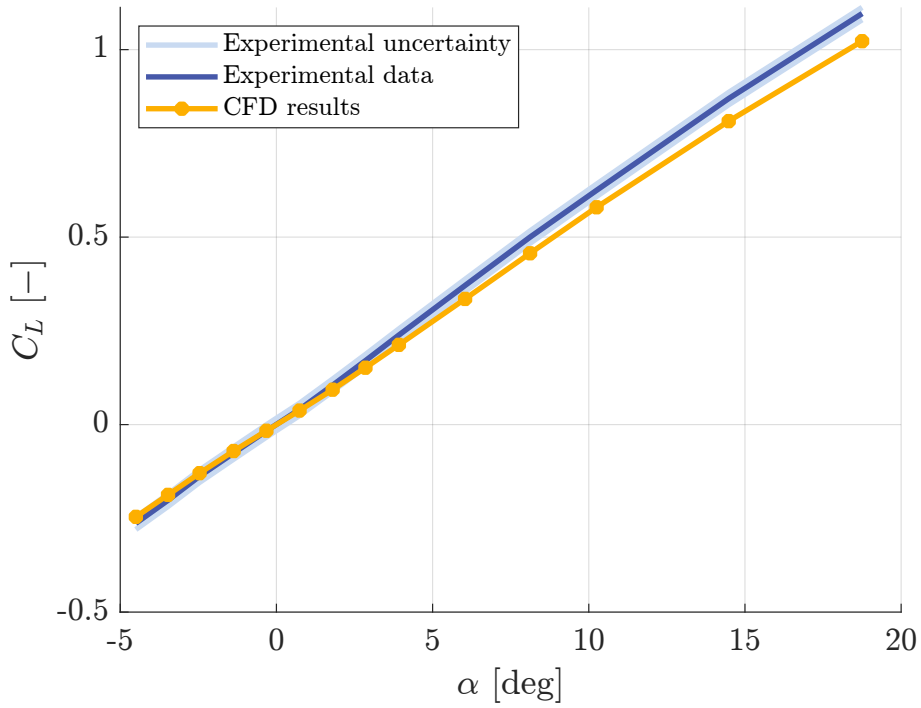


Figure 22: Comparison of lift coefficient C_L versus angle of attack at $M = 1.0$.

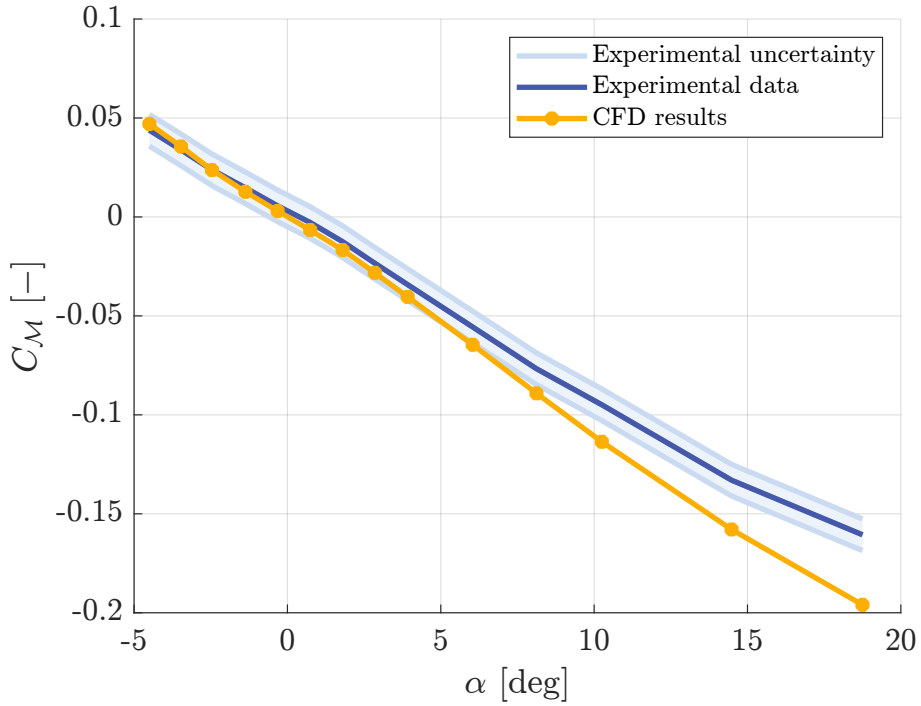


Figure 23: Comparison of pitching moment coefficient C_M versus angle of attack at $M = 1.0$.

- The numerical model captures both linear and nonlinear aerodynamic behavior.
- The agreement remains consistently good from low-subsonic to transonic conditions.

The residual discrepancies are compatible with the expected uncertainties associated with steady RANS turbulence modeling, mesh discretization, and experimental measurements.

E.10 Conclusions

The validation campaign confirms that the CFD methodology adopted in this work provides reliable aerodynamic predictions over the range of flow conditions relevant to the present study.

The close agreement with benchmark wind tunnel data provides strong support for the use of the numerical setup described in Appendix D for the generation of the aerodynamic datasets employed for surrogate modeling and flight dynamics analyses.

In particular, the validation demonstrates that:

- The pressure-based coupled solver is robust for compressible aerodynamic simulations.
- The k–ω SST turbulence model provides accurate force and moment predictions.
- The mesh strategy with wall-resolved inflation layers is adequate.
- The overall methodology yields aerodynamic coefficients of sufficient accuracy for engineering applications.

References

[1] C. F. Anderson. An investigation of the aerodynamic characteristics of the agard model b for mach numbers from 0.2 to 1.0. Technical Report AEDC-TR-70-100, Arnold Engineering Development Center, Tullahoma, Tennessee, USA, 1970.

[2] ANSYS Inc. *ANSYS Fluent Theory Guide*. ANSYS Inc., 2025. Release 2025 R2.

[3] ANSYS Inc. *ANSYS Fluent User's Guide*. ANSYS Inc., 2025. Release 2025 R2.

[4] James S Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pages 2546–2554, 2013.

[5] Dheeru Dua and Casey Graff. Uci machine learning repository. `http://archive.ics.uci.edu/ml`, 2019.

[6] Alexander I. J. Forrester, András Sóbester, and Andy J. Keane. *Engineering Design via Surrogate Modelling: A Practical Guide*. John Wiley & Sons, Chichester, UK, 2008.

[7] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

[8] Robert B Gramacy. *Surrogates: Gaussian process modeling, design, and optimization for the applied sciences*. CRC press, Boca Raton, FL, 2020.

[9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[10] Diederik P Kingma and Jimmy Lei Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[11] Günter Klambauer, Thomas Unterthiner, Andreas Daxberger, and Sepp Hochreiter. Self-normalizing neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 30:971–980, 2017.

[12] Yann LeCun, Léon Bottou, Genevieve B Orr, and Klaus-Robert Müller. Efficient backpropagation. In *Neural networks: Tricks of the trade*, pages 9–50. Springer, 1998.

[13] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*, 2019.

[14] Diganta Misra. Mish: A self regularized activation function. *arXiv preprint arXiv:1908.03682*, 2019.

[15] Christopher J Paciorek and Mark J Schervish. Nonstationary covariance functions for gaussian process regression. *Advances in Neural Information Processing Systems (NeurIPS)*, 16:273–280, 2004.

[16] Lutz Prechelt. Early stopping-but when? *Neural Networks: Tricks of the trade*, pages 55–69, 1998.

[17] Carl Edward Rasmussen and Christopher KI Williams. *Gaussian processes for machine learning*. MIT press, Cambridge, MA, 2006.

[18] Andrew Gordon Wilson, Zhiting Hu, Ruslan Salakhutdinov, and Eric P Xing. Deep kernel learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, pages 6849–6858, 2016.